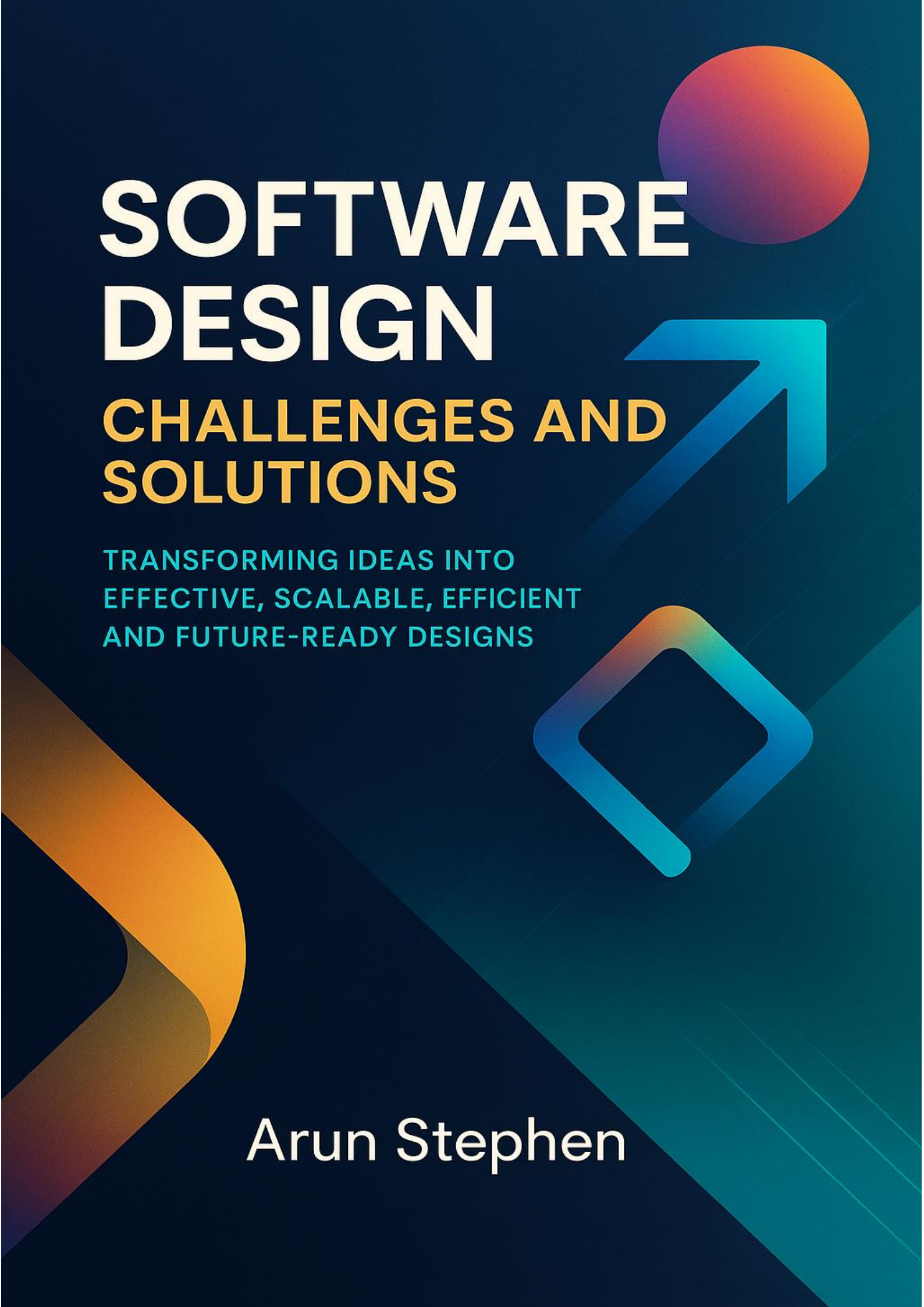




SOFTWARE DESIGN

CHALLENGES AND SOLUTIONS

TRANSFORMING IDEAS INTO
EFFECTIVE, SCALABLE, EFFICIENT
AND FUTURE-READY DESIGNS

Arun Stephen

It captures my journey in transforming a project into an enterprise-standard application. As a Technology Architect, I analysed the existing architecture, infrastructure, codebase, and design issues to identify pain points and deliver effective, scalable solutions with automation, security enforcement, and architectural improvements.

I have designed and implemented end-to-end solutions, addressing challenges across UI, backend, database, and cloud infrastructure, while ensuring alignment with client and project constraints. Although UI mock-ups are not included here, I provide detailed explanations during technical discussions when required. My involvement extended beyond design to complete code implementation, deployment, and production readiness.

The process required deep analysis, iterative design, and careful evaluation of alternatives. This document highlights the final selected solutions—supported by diagrams, problem statements, technology choices, and their benefits. The approach emphasizes scalability, security, and efficiency, while balancing business needs and technical constraints.

Beyond project-specific solutions, my contributions extend to improving processes, quality, security, and productivity. I have prepared architectural diagrams, technical documentation, developer guidelines, and security best practices. The proposed solutions, successfully adopted by both internal and external teams, have been implemented in production with measurable success.

Table of Content

1. [Project Critical Challenges & Limitations](#)
2. [Modernized Backend Architecture with Secure, Scalable Enhancements](#)
3. [On-boarding New Application](#)
4. [Zero Trust Security Implementation](#)
5. [Chatbot Integration](#)
6. [NGCGI Integration – Automated Compensation Assignment & Goal Mapping](#)
7. [Automation of Updating Data using Python Module](#)
8. [Generate Worker Council Report using Python Module](#)
9. [Award Event Integration Modernization and Automation](#)
10. [Optimization of Static File Management in Frontend Applications](#)
11. [Requirement Specification: Document Upload and Review Workflow](#)
12. [Requirement Change: Hierarchical Comment Tracking & API Optimization](#)
13. [Optimizing Branching Strategy for CI/CD and Environment Management](#)
14. [Optimizing Rate Details Management and Query Performance](#)
15. [Application Usage Monitoring and Metrics Dashboard](#)
16. [External Application Integration via Secure API Proxy](#)
17. [Centralized Cache Optimization with Redis](#)
18. [Cost Optimization and Technology Modernization Initiatives](#)
19. [Conclusion – Enterprise-Grade Application Transformation](#)

1. Project Critical Challenges & Limitations

Upon joining the project as Technology Architect, I conducted a detailed review of the architecture, codebase, and delivery processes. The assessment revealed several critical challenges impacting scalability, security, and maintainability.

Key Challenges Identified

- **Code Quality & Maintainability**
 - Lack of modular structure and reusable components: increased technical debt and onboarding difficulty.
 - Hardcoded business logic and redundant code: poor scalability and higher defect risks.
 - No environment-specific configurations: inconsistent deployments across environments.
- **Logging & Monitoring**
 - Inconsistent, non-centralized logging: difficult incident tracing and delayed resolution.
 - No database monitoring or alerting: hidden performance bottlenecks and risks.
 - Limited debugging support: inefficient troubleshooting.
- **Security & Authentication**
 - Weak OAuth2/Implicit flow implementation: exposure to unauthorized access.
 - Some APIs lack authentication layers: major security vulnerabilities.
 - Missing secure integration mechanisms (e.g., Webex, email): restricted collaboration.
- **API & Integration**
 - No API governance or standardization: lack of version control, scalability, and lifecycle management.
 - System operates in isolation: poor interoperability with external/enterprise services.
- **Technology Choices**
 - Node.js with raw JavaScript: runtime errors, weak typing, and low developer productivity.
 - PostgreSQL community edition without enterprise features: limited scalability, HA, and support.
- **Automation & Operations**
 - Manual deployments, testing, and monitoring → slow delivery, inconsistency, and reduced release confidence.
 - Absence of CI/CD pipelines and automated test suites → poor maintainability and scalability barriers.

Summary of Risks

The project currently faces fundamental gaps across **architecture, security, scalability, and operations**, creating risks in:

- **Maintainability:** High technical debt due to poor coding practices.
- **Scalability:** Lack of modularity and code reuse block future growth.
- **Security:** Weak authentication and missing API governance expose vulnerabilities.
- **Reliability:** Limited monitoring, automation, and structured workflows weaken production stability.
- **Interoperability:** Absence of external integrations restricts ecosystem readiness.

2. Modernized Backend Architecture with Secure, Scalable Enhancements

I led the transformation of the backend to enterprise-grade standards by addressing architectural gaps with robust, scalable, and secure solutions. The modernization effort focused on improving scalability, security, maintainability, and developer productivity through the following key initiatives:

1. Backend Modernization & Productivity

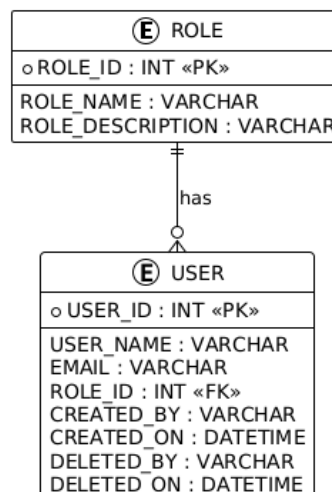
- ✓ Rebuilt the backend in **TypeScript** for strong typing and maintainability.
- ✓ Standardized code formatting with **Prettier** and package management via **Yarn**.
- ✓ Modularized the codebase for reusability and reduced complexity.
- ✓ Introduced **environment-specific** configurations with secure credential handling using Triple **DES** encryption.

2. Environment Replication for Debugging

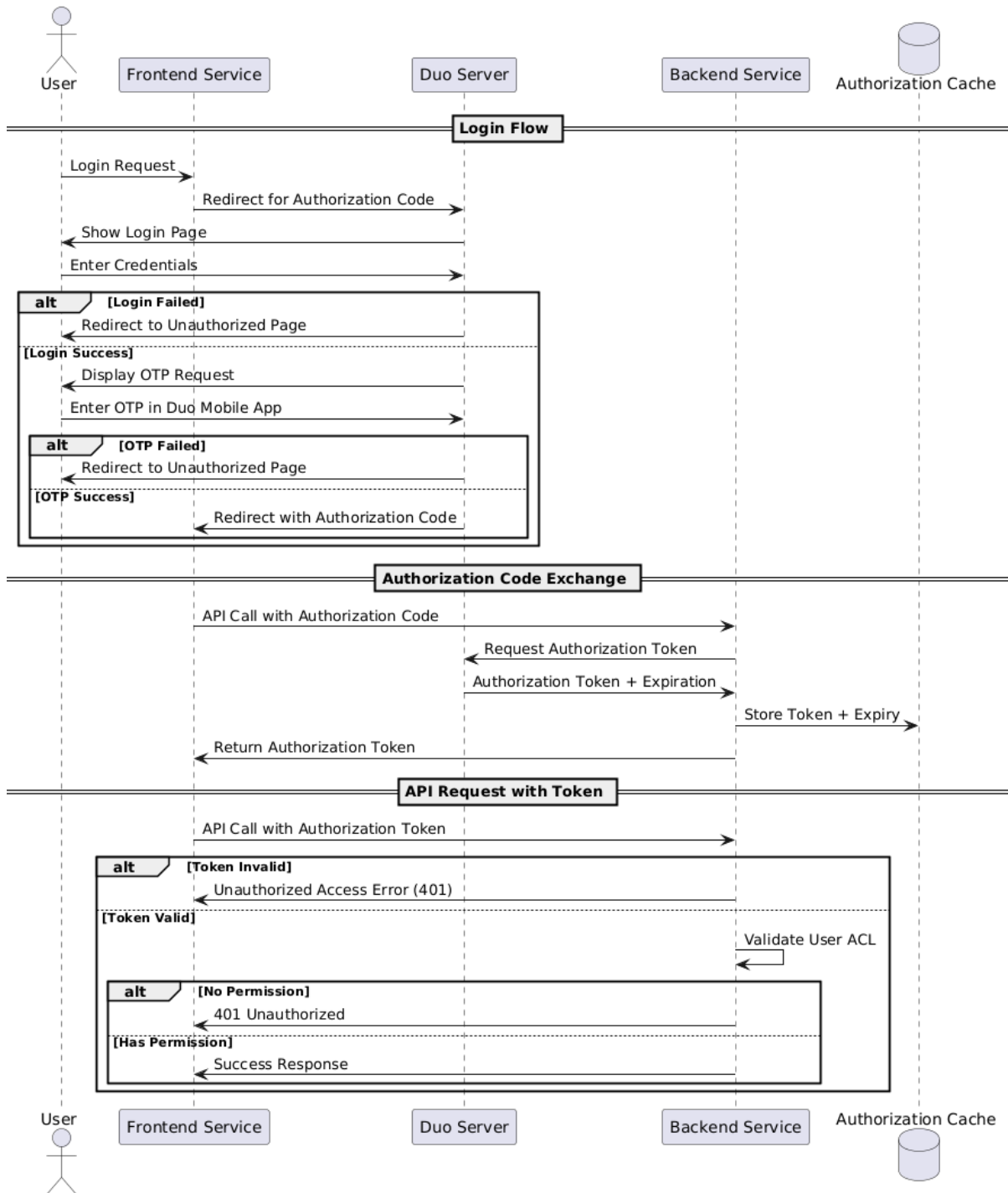
- ✓ Enabled local execution of environment-specific builds.
- ✓ Allowed developers to replicate production issues locally, accelerating resolution cycles.

3. Advanced Authentication & Authorization

- ✓ Designed **RBAC-based access control** with granular role/permission mappings.
- ✓ Optimized schema from multiple tables to a lightweight **USER-ROLE** model.
- ✓ Introduced an **ACL API** for dynamic role assignment and secure API-level access.
- ✓ Strengthened security with **Duo MFA** for two-factor authentication.



The below diagram illustrates the implementation of DUO and MFA authentication and authorization, providing a clear end-to-end understanding of the workflow.



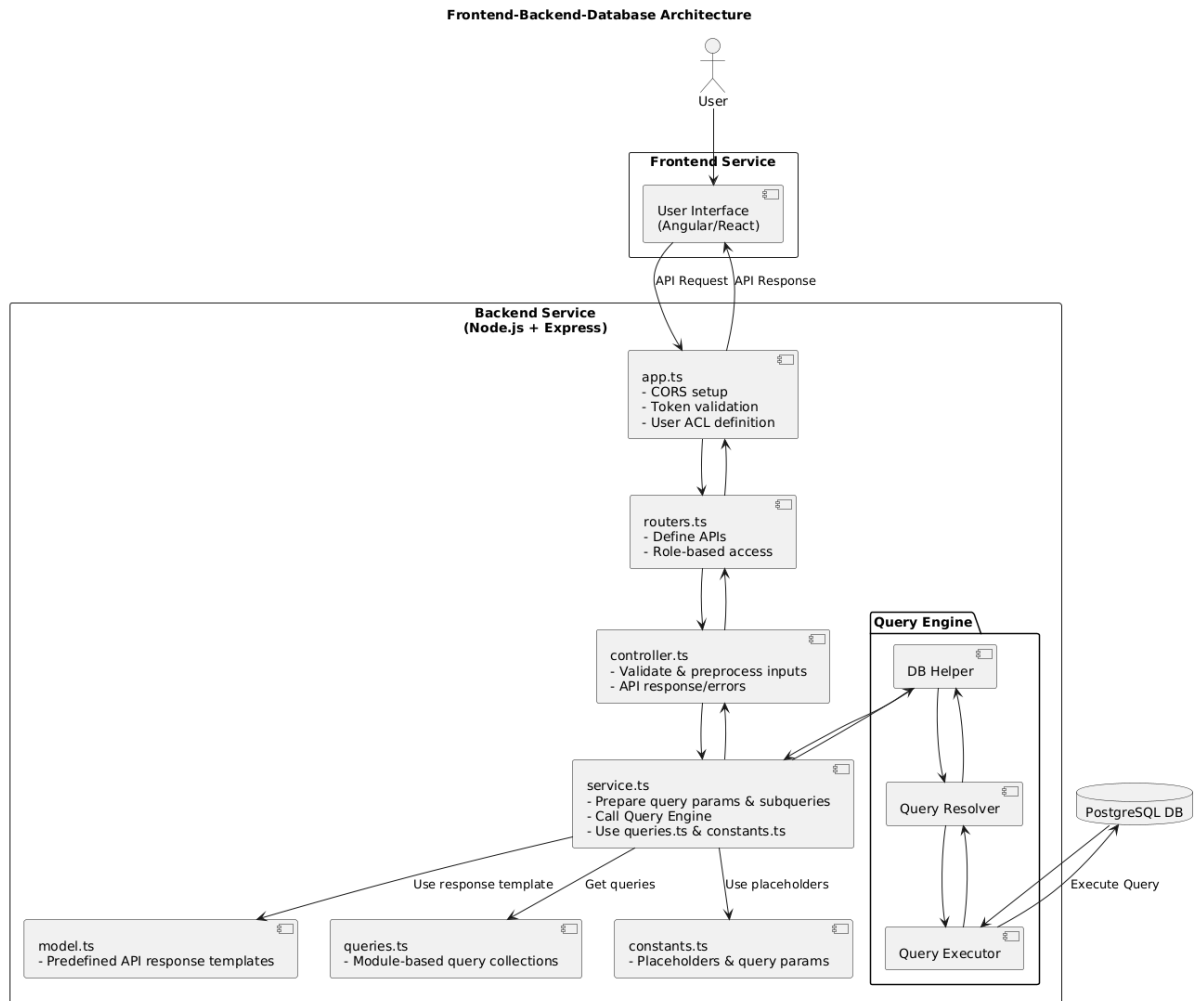
4. Data Security & Auditability

- ✓ Implemented **database-level monitoring** with full audit trails.
- ✓ Added **tracking fields** (CREATED_BY, UPDATED_BY, etc.) across critical tables for accountability.

5. Centralized Query & Validation Layer

- ✓ Built a **centralized query resolver** for consistent and high-performance database interactions.
- ✓ Enforced **gateway-level request validation** with role-based API security.

The diagram below illustrates the complete project architecture, highlighting the detailed flow of the backend service.



6. Scalable Features & User Experience

- ✓ Added **streaming support** for large responses and file downloads.
- ✓ Enabled **real-time notifications** via Webex, email, and other channels.

7. Containerization & Deployment

- ✓ Enhanced the **Dockerfile** to compile TypeScript during build.
- ✓ Deployed the application as a **containerized** image, ensuring portability and streamlined CI/CD.

3. On-boarding New Application

Problem Statement

The current application is managed by Medahub (third-party vendor) and hosted outside Cisco's on-premises infrastructure. It follows a monolithic architecture, where data exchange, business logic, database access, and HTML rendering are tightly coupled on the server.

Due to vendor restrictions, Cisco has limited visibility into application logic, database, and file access. The solution also incurs high operational costs and raises data security concerns as sensitive information is shared externally. Furthermore, Medahub integrates with event management teams but provides no knowledge transfer or implementation insights.

Expected Outcome

Cisco plans to replicate the application in-house with no loss of functionality while ensuring:

- **Data security and compliance** within Cisco's infrastructure.
- **Elimination of third-party dependency** and reduced operational cost.
- **Full transparency and control** over logic, database, and architecture.
- **Seamless integration** with event management processes.

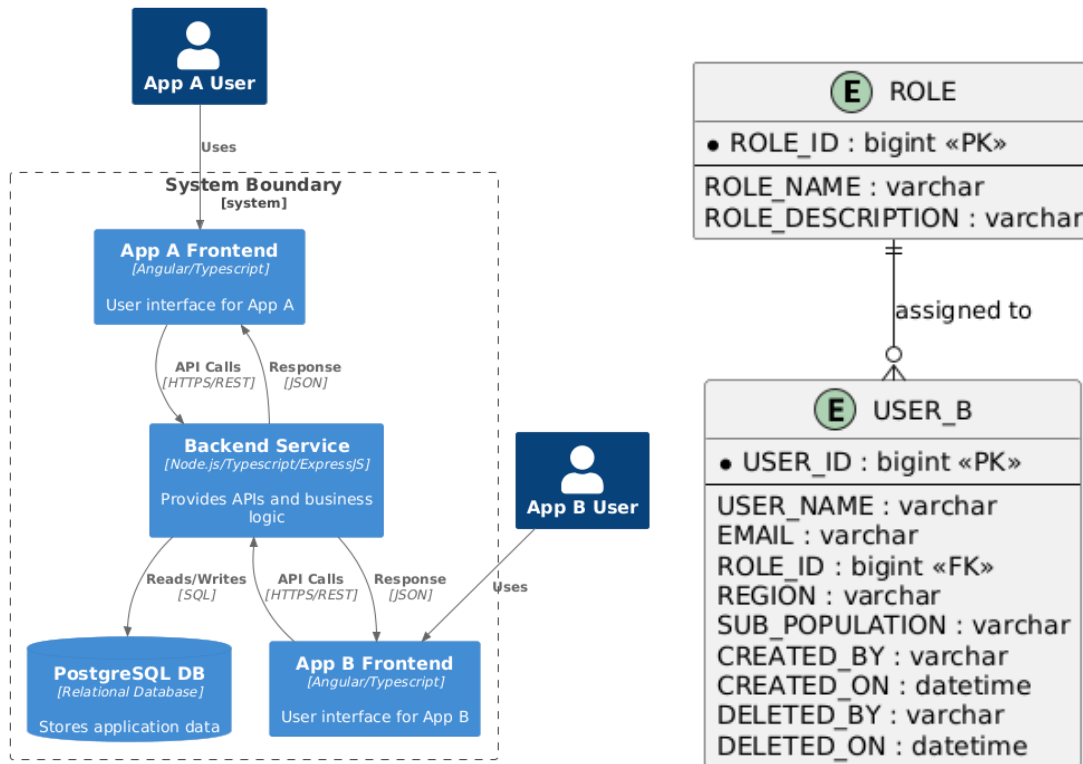
Proposed Technical Stack

- **Frontend:** Angular, TypeScript
- **Backend:** Node.js, TypeScript, Express.js
- **Database:** PostgreSQL

Design Highlights:

- Only the **Frontend service** requires onboarding to Cisco on-premises; other services are already deployed.
- **Database tables** provisioned with proper separation and relevant data population.
- **APIs** developed for smooth communication between Frontend and Backend.
- **Backend enhancements** implemented for CORS handling, token validation, and route management to support multiple applications

The below architecture diagram illustrates multiple frontends sharing a common backend and database, with a unified user authentication table.



4. Zero Trust Security Implementation

Before onboarding any application into production, the following security and compliance requirements must be met:

1. Centralized Logging & Retention

- ✓ Retain production logs securely for at least 6 months.
- ✓ Ensure logs support complete user activity tracking for audit and monitoring.

2. Database Security

- ✓ Community edition databases must not be used for storing PII in production.
- ✓ Only enterprise-grade, security-compliant databases are permitted.

3. Application Security Scanning

- ✓ Perform comprehensive application security scans.
- ✓ Remediate all critical, high, and medium vulnerabilities before go-live.
- ✓ Maintain updated scan results in the Enterprise Security Portal (ESP).

4. Secure Access & Proxy Management

- ✓ Enable DNG Proxy service and externalize application domains to ensure secure, VPN-independent access.

Tools Utilized:

- **Centralized Logging & Analysis:** Splunk
- **Database Platform:** Snowflake
- **Frontend Reverse Proxy & Secure Access:** NGINX

Design Decisions – Logging, Security & Data Migration

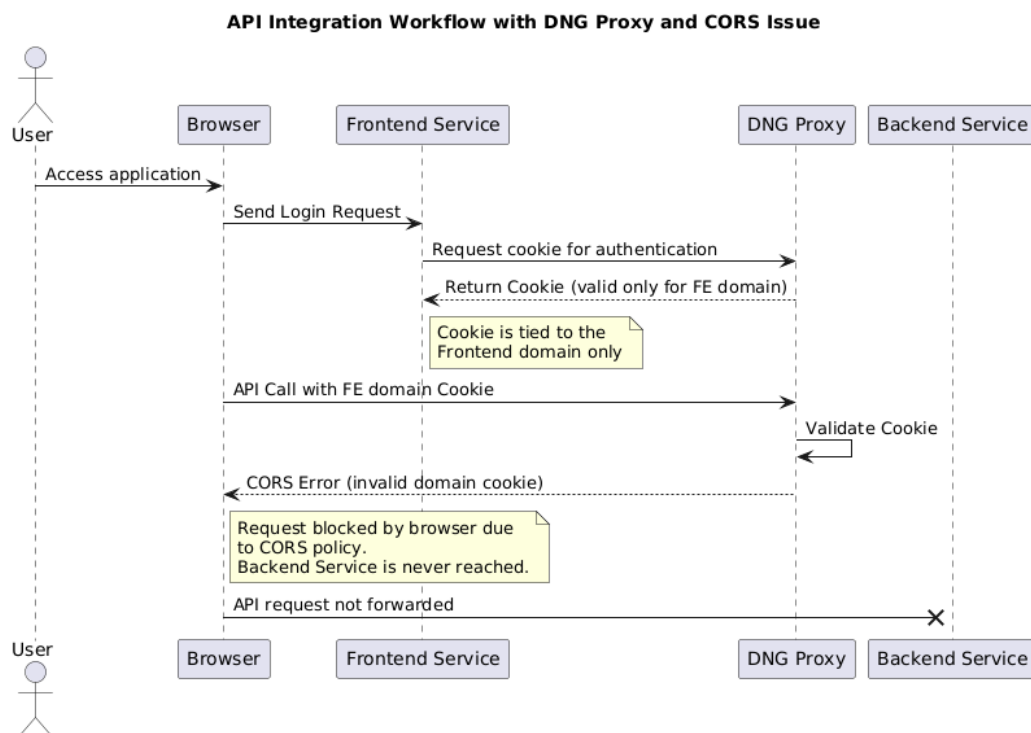
1. Centralized Logging

- Implemented singleton application-level logging for consistency across log levels.
- Captured key details: timestamp, client IP, user identity, API endpoint, headers, and response time.
- Environment-specific log levels:
 - Production → **INFO**
 - Lower environments → **DEBUG**
- Enabled full query logging in Development/Local for direct debugging.
- Integrated with Splunk for 1-year log retention and enhanced observability.

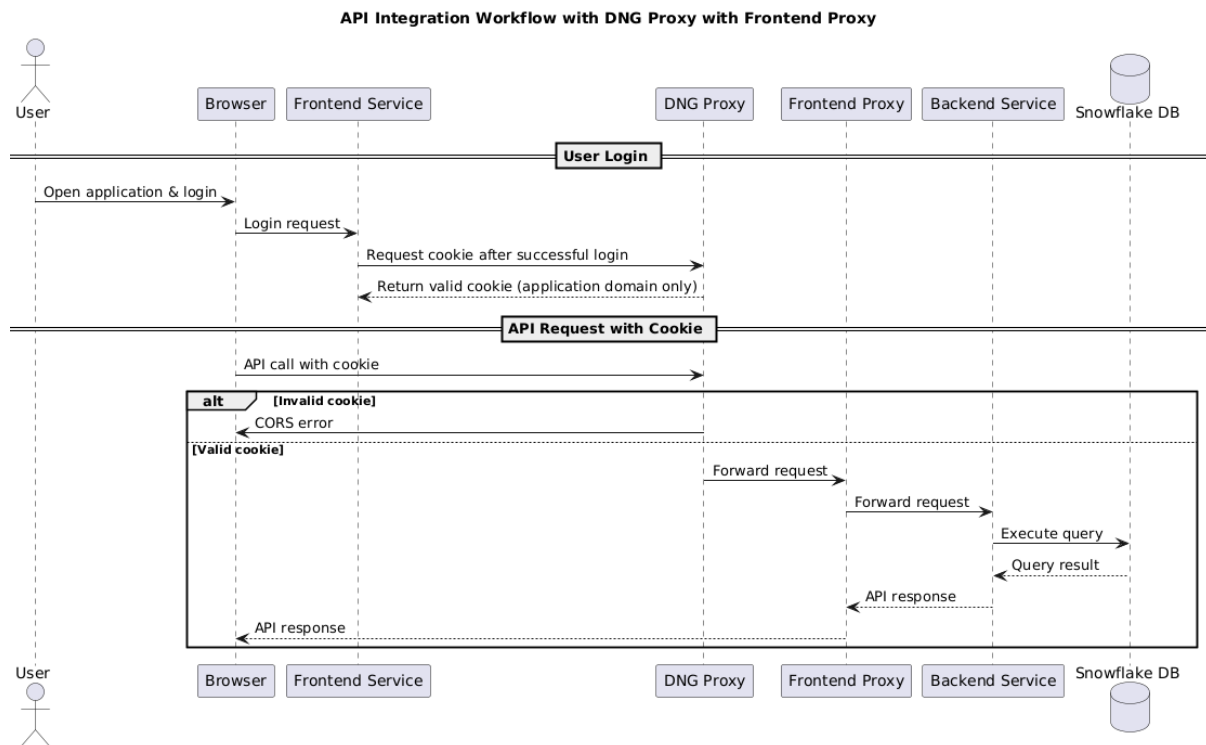
2. Frontend Security & API Access

- Applied reverse proxy on Frontend Service to securely route and validate all API requests to Backend Services.

The diagram illustrates the API workflow via the DNG proxy, highlighting that direct access from the frontend service (without the proxy or VPN) results in a CORS error.



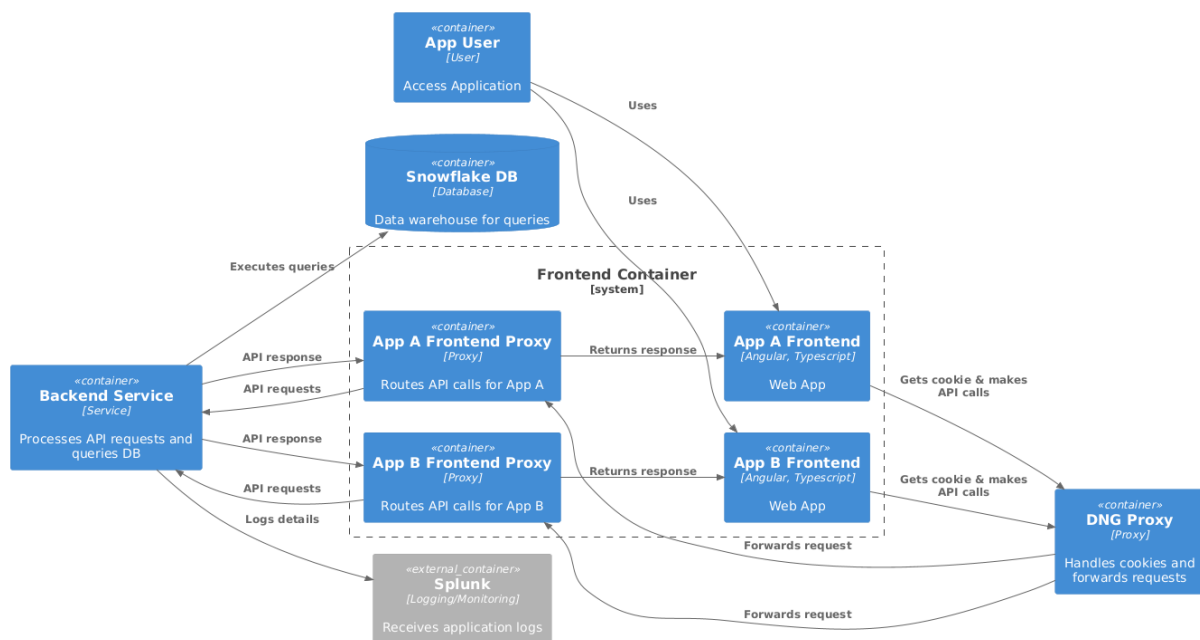
The diagram illustrates the API workflow via the DNG proxy, enabling secure access without VPN and resolving CORS issues through the frontend proxy server.



3. Database Migration & Query Handling

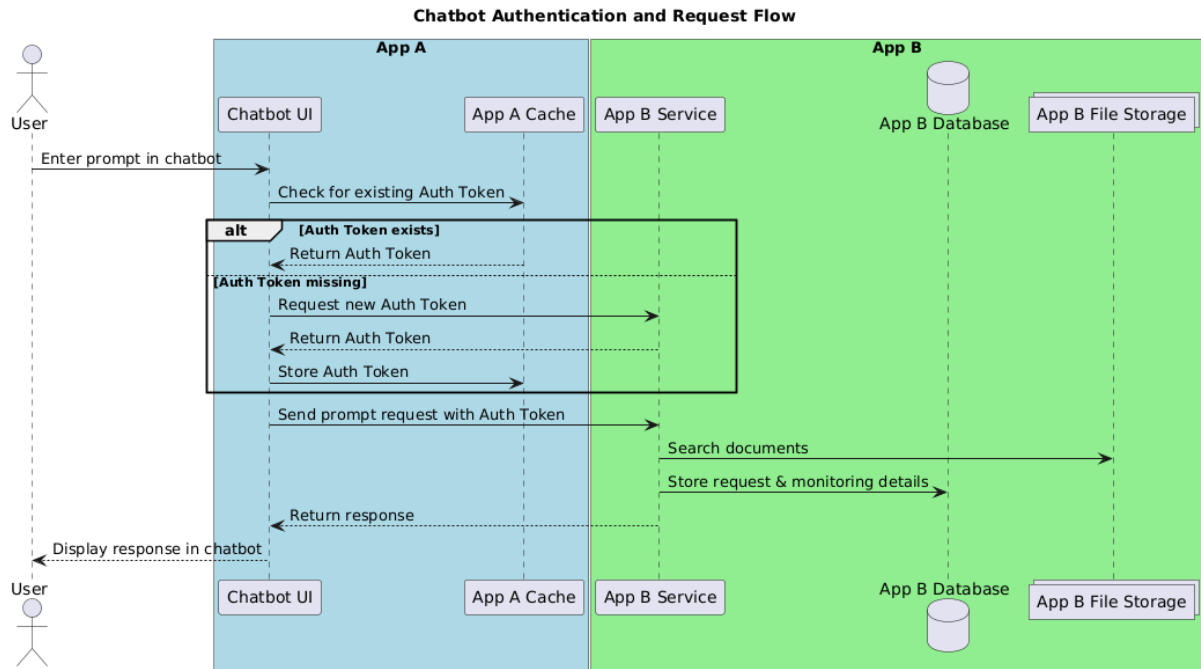
- Successfully migrated from PostgreSQL to Snowflake for secure, scalable data management.

The diagram illustrates the complete project architecture after migrating the database from PostgreSQL to Snowflake, integrating centralized logging with Splunk, and adding a proxy server for frontend services.



5. Chatbot Integration

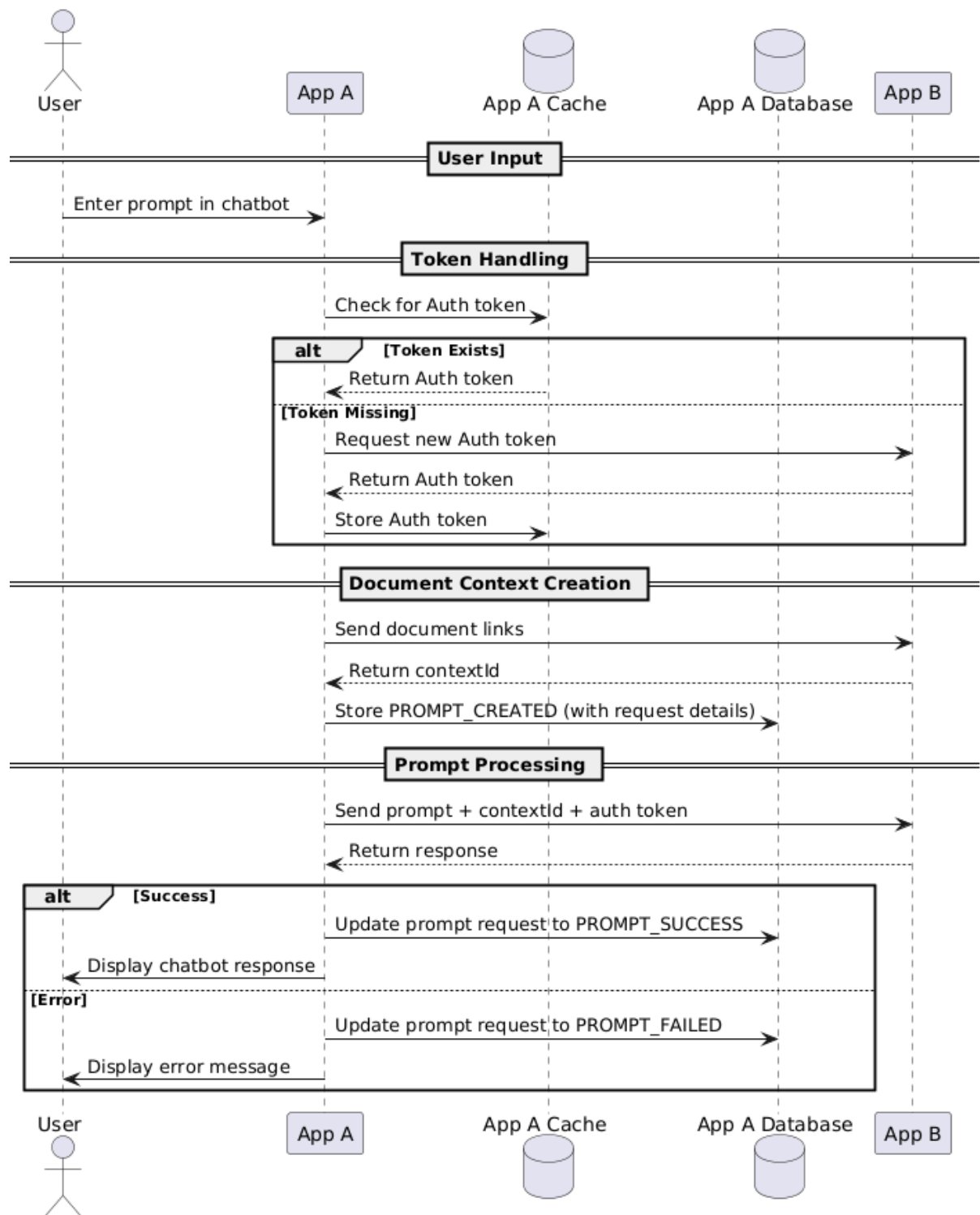
The chatbot integration was originally designed by an external team for their application. The client requested us to leverage the same services and integrate them into our application. Based on the shared design, we implemented the solution within our project. The initial proposed chatbot integration design is illustrated below.



Following the successful chatbot integration, I identified design concerns in the external team's proposal and recommended an improved design approach to reduce complexity and ensure better alignment with the external application. I highlighted the following design challenges and concerns for the team's technical discussion.

1. **Tight Coupling with External Systems:** Current design depends on external teams for document uploads into their storage systems, creating tight coupling, reduced flexibility, and higher coordination overhead.
2. **Scalability Challenges:** The architecture is not easily extensible. As integrations increase, scaling the solution becomes complex and costly.
3. **Redundant Document Sharing:** Documents must be repeatedly shared across teams to update their storage, causing inefficiencies and reducing search accuracy.
4. **Security & Compliance Risks:** Storing monitoring details in external applications raises concerns around data security and regulatory compliance.
5. **Resource & Performance Constraints:** With more integrated applications, database management and network bandwidth demand intensify, risking performance degradation and reduced reliability.

The below diagram illustrates the end-to-end chatbot integration workflow, clearly defining application boundaries and their operational roles.

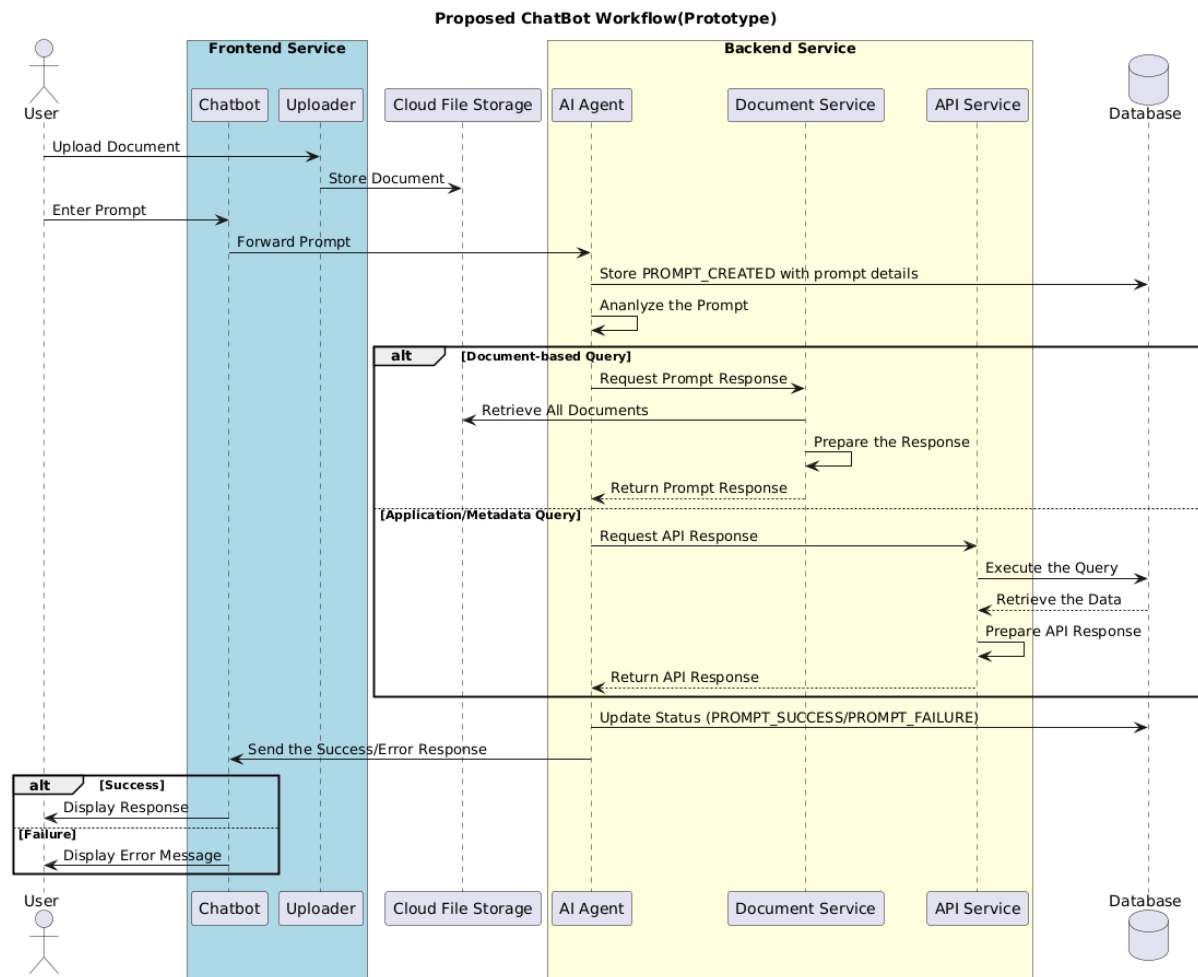


Proposed Solution Overview

Our solution leverages an AI-driven agent to address key challenges with the following capabilities:

- **Document Management** – Seamless integration and management of application documents.
- **Adaptive Logic** – Flexible business logic fine-tuning to meet evolving needs.
- **Monitoring & Insights** – Built-in tracking and governance through real-time monitoring.
- **Intelligent Q&A** – Document-based question-answer support for faster knowledge access.
- **API Integration** – Unified, user-friendly view by consolidating data from external APIs.

The below diagram illustrates a proposed design for integrating a complete chatbot within the application using an AI Agent, eliminating the need for external applications. This solution is not yet scheduled for implementation but is planned for future consideration.



6. NGCGI Integration – Automated Compensation Assignment & Goal Mapping

Problem Statement:

The current seller compensation and goalsheet mapping process via CGI is highly manual and error-prone. It requires repetitive Excel downloads, updates, and exchanges with the CGI team, leading to:

- Heavy manual intervention to track updates
- Frequent full reprocessing instead of incremental changes
- Operational inefficiency and delays in reflecting assignments

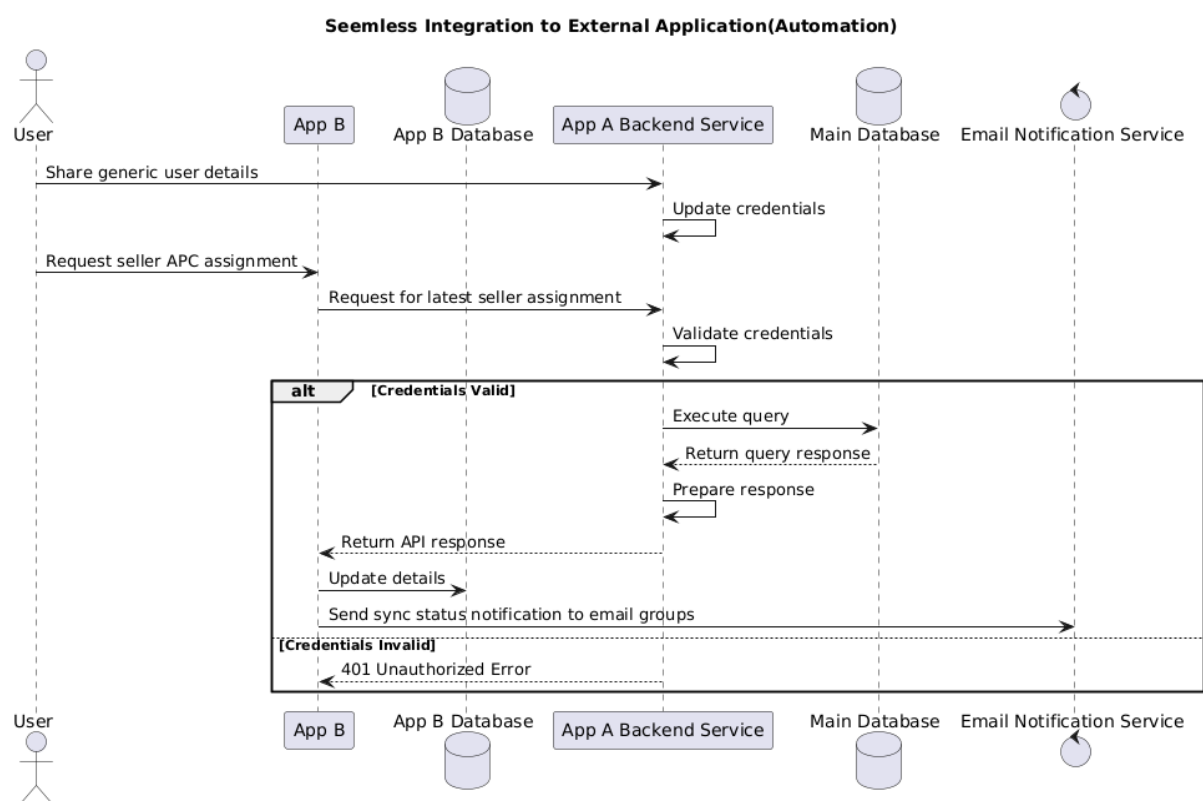
Proposed Solution – Automation through CGI Integration:

To eliminate manual effort and ensure timely updates, an automated integration with CGI is recommended:

1. **API Enablement:** Develop an SCP-based API exposing mapping data by Year, Plan Type, and Time Range.

2. **Secure Authentication:** Integrate via a generic system user for consistent and secure access.
3. **Time Synchronization:** Use UTC timestamps to ensure uniform record updates across time zones.
4. **Automated Sync:** Schedule updates every 4 hours to keep CGI data aligned with the latest mappings.
5. **Error Handling & Resync:** Track sync status per seller (via CGI_SYNC_AT column) to allow selective re-sync of failed records.
6. **Notifications:** Provide stakeholders with real-time status and error alerts through email notifications.

The below diagram illustrates automated integration that retrieves the latest assignment updates at scheduled intervals and seamlessly synchronizes them with the target application, eliminating manual effort.



Additional Gap Identified

Currently, users lack visibility into sync status and must raise support requests for clarification. This requires cross-team analysis to determine the actual sync state and resolve reported issues, resulting in delays and inefficiencies.

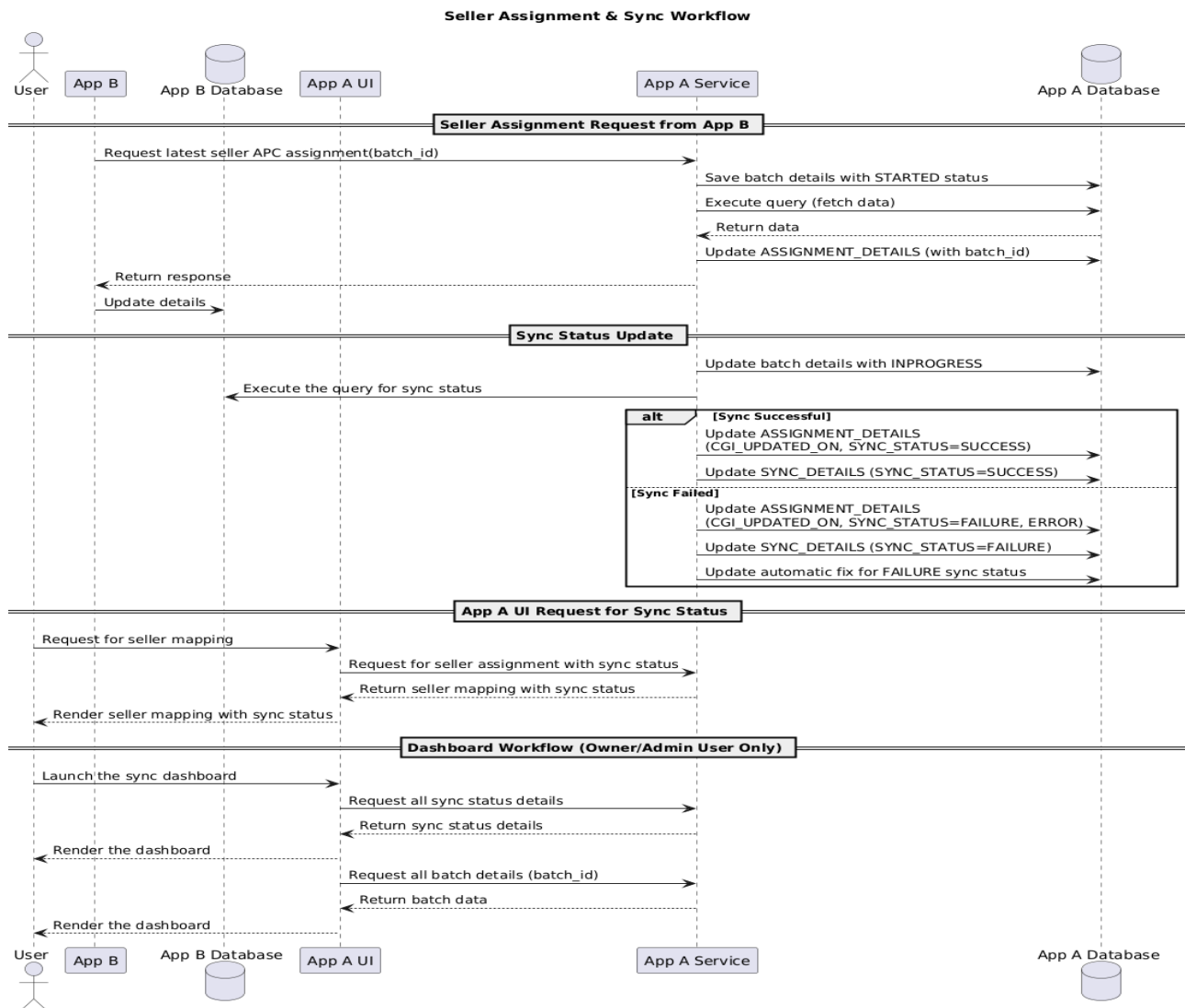
Problem Areas

- Sellers cannot view sync status in the UI, leading to repeated support queries.
- Support teams depend on direct database checks, causing delays and team dependencies.
- Failure cases require manual intervention, with no automated handling of external or use-case-driven issues.

Proposed Solution – Automation & Visibility

- Extend existing tables to capture sync status.
- Integrate with the CGI database to fetch sync details.
- Update and display sync status in the SCP application UI with clear success (✓) and failure (✗) indicators.
- Implement automated retries for failures in subsequent cycles.
- Eliminate email-based notifications.
- Introduce a new dashboard to validate sync status and trigger re-sync for specific seller mappings.
- Restrict dashboard access to Owner/Admin roles.

The below diagram illustrates an enhanced approach for displaying the mapping sync status directly in the UI, eliminating the need to engage with support or other teams.



7. Automation of Updating Data using Python Module

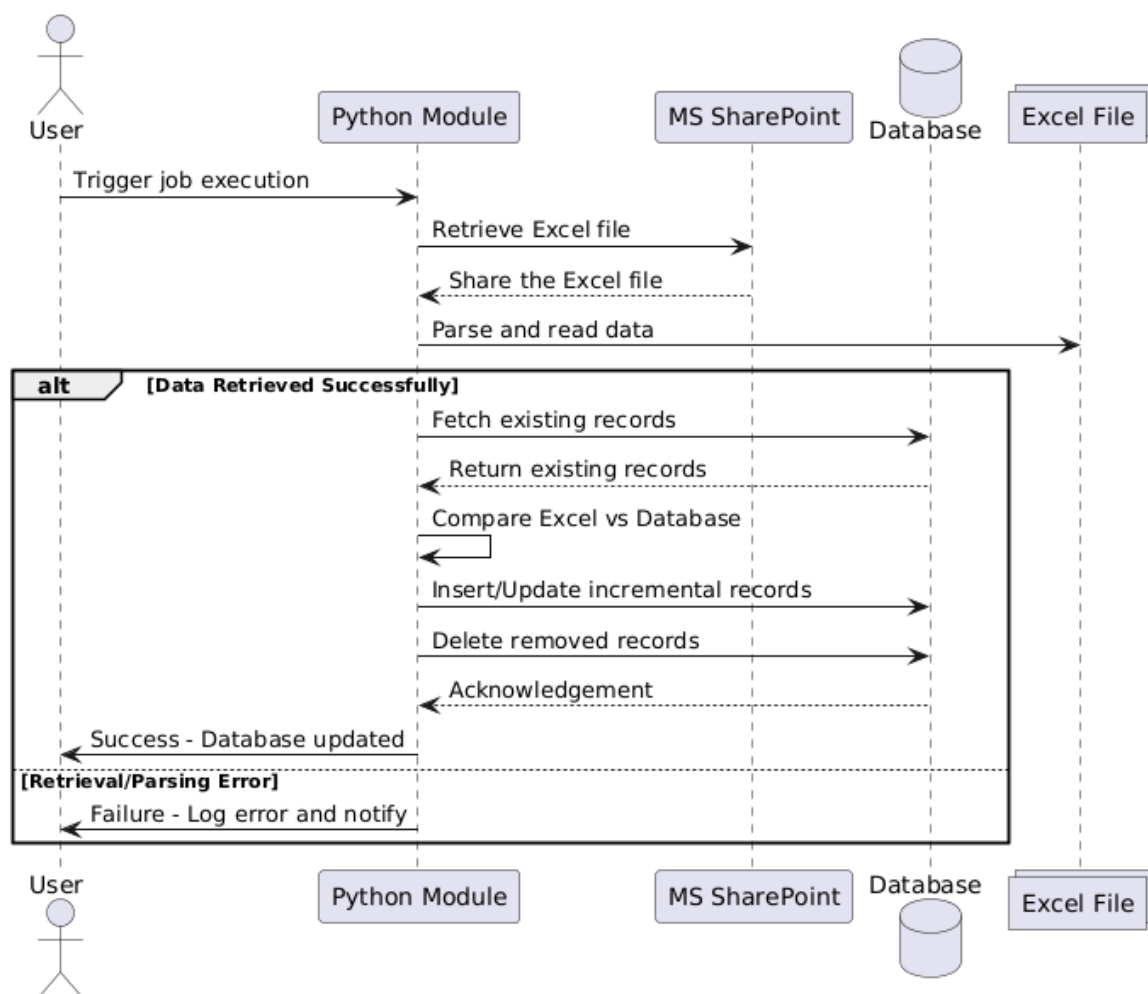
Problem Statement:

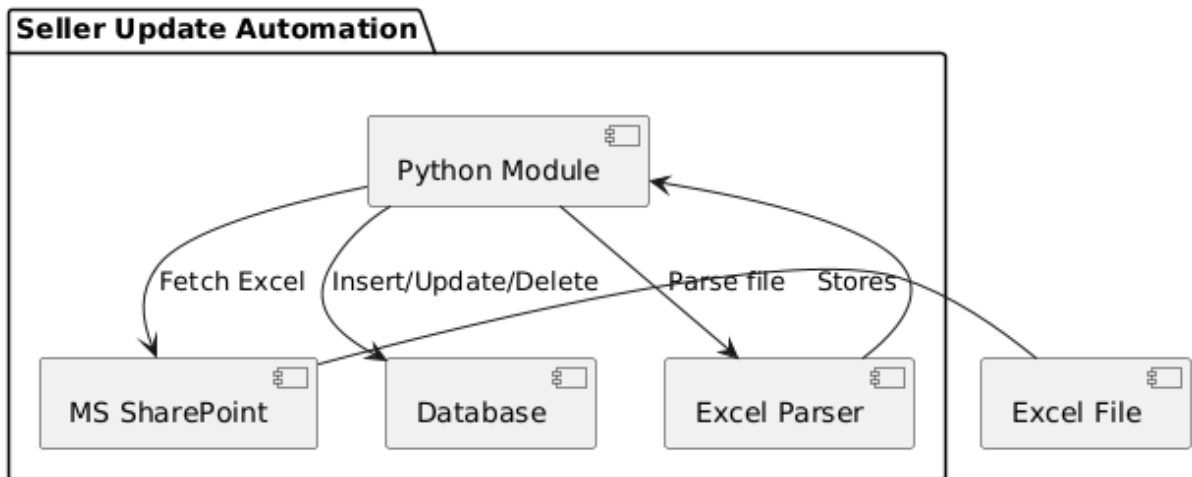
The existing process of updating seller hierarchy (additions, removals, or structural changes) relied on manual follow-ups and Excel sheets. The workflow was managed through Alteryx, which incurred high licensing costs and required significant manual effort to update daily records into the database.

Initial Solution (Automation):

- **Data Handling:** Sellers upload Excel sheets directly into Microsoft SharePoint.
- **Automation Module:** Developed a Python module from scratch with support for dependency resolution, dynamic environment-based configurations, and scheduled background jobs.
- **Deployment:** Containerized the solution using Docker and deployed it to cloud infrastructure.
- **Monitoring:** Automated email notifications triggered upon job completion, indicating **SUCCESS** or **FAILURE**.

The below diagram illustrates the automation workflow for updating data in the database.

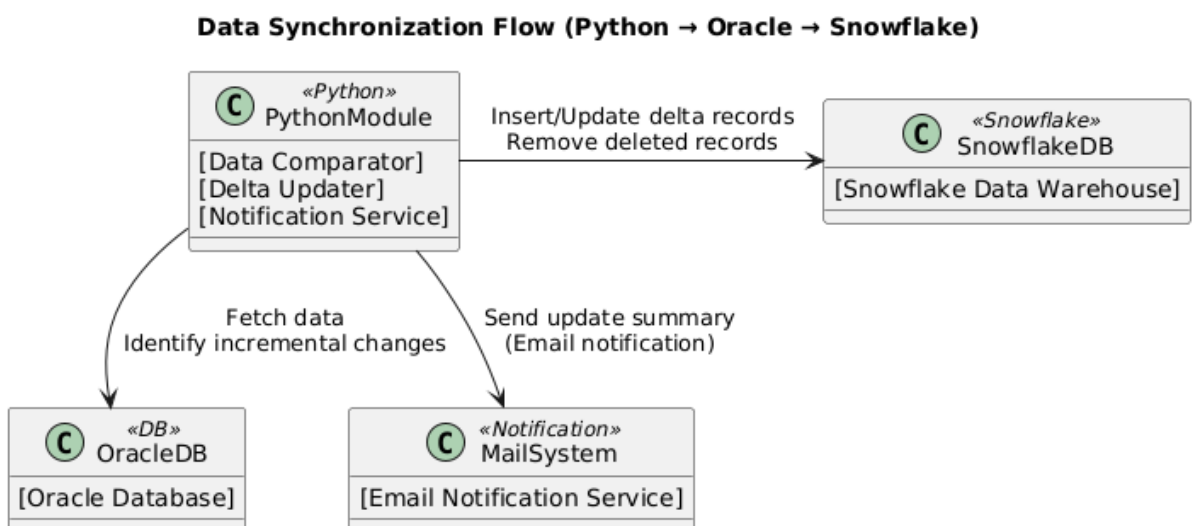




Actual Solution (Automation):

- Replaced Excel and Microsoft SharePoint dependency with direct Oracle DB integration.
- Implemented environment-specific Oracle configurations.
- Established Oracle connectivity for dependency management, query execution, and data handling.
- Updated relevant details in the application database.
- Automated email notifications to indicate **SUCCESS** or **FAILURE** status.

The below diagram illustrates the extended process of collecting data from the source and updating it into the database.



8. Generate Worker Council Report using Python Module

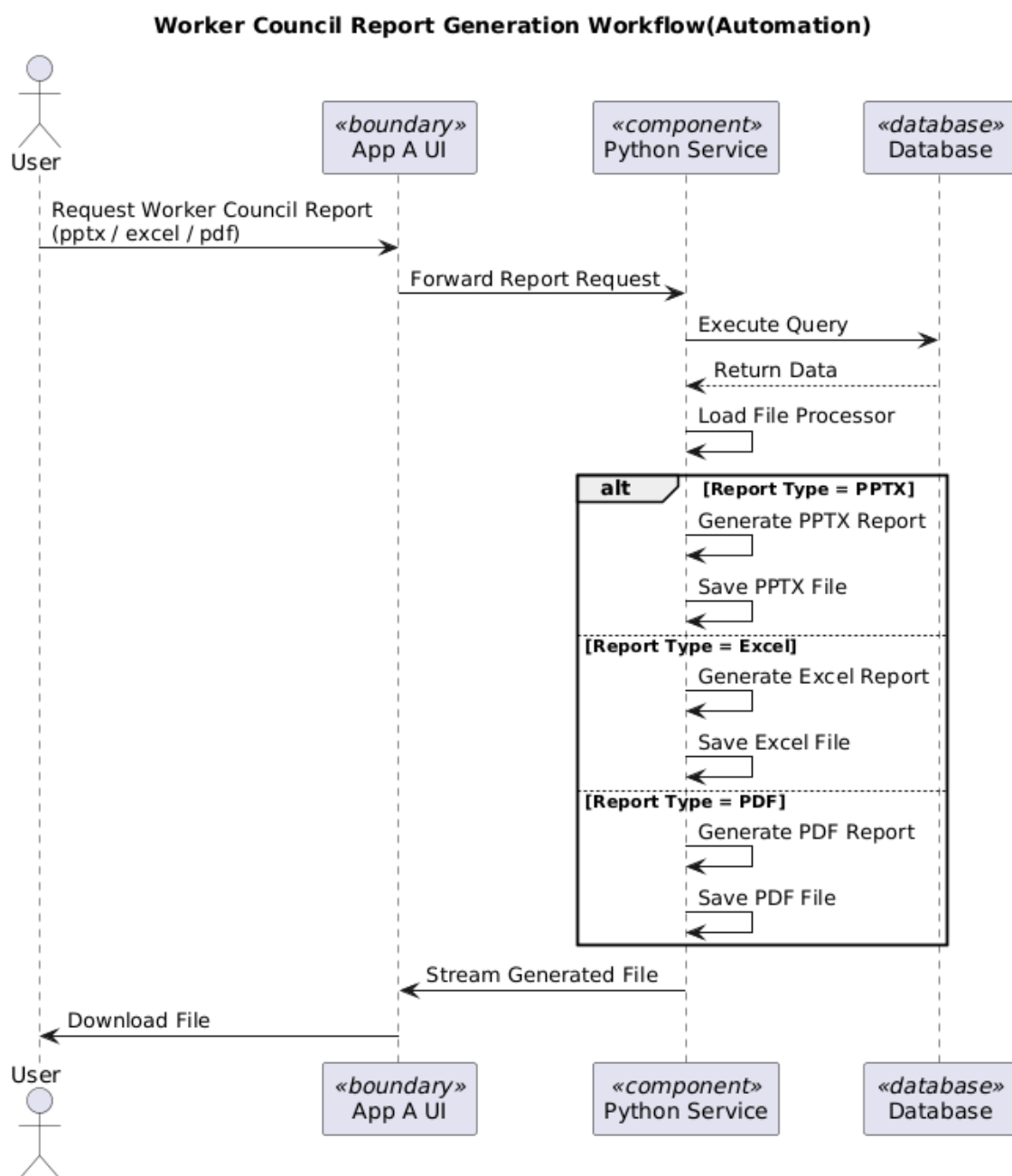
Problem Statement:

Manual preparation of Worker Council files (**PPTX, Excel, PDF**) consumes **2–4 weeks per release**, impacting team productivity. This repetitive effort recurs for every relevant release and senior management discussion.

Proposed Solution:

- Introduced **API support** within the Python module.
- Integrated dependencies for multi-format file generation (**PPTX, Excel, PDF**).
- Applied **Strategy** and **Template Design Patterns** for extensibility and maintainability.
- Implemented **server-side directory management** for file generation.
- Enabled **streaming support** for large file downloads.
- Designed solution for **dynamic period file generation** without code changes.
- Provided **API integration** with existing applications.
- Enhanced **UI with download capability** for end users.

The below diagram below illustrates the process of generating the worker council report using a Python module.



9. Award Event Integration Modernization and Automation

Problem Statement:

RainFocus, the event management partner, conducts award events based on company-provided data. The company shares details of eligible candidates, and RainFocus manages seller data collection, sends event notifications, and facilitates in-person participation for award distribution. However, RainFocus currently relies on a **legacy integration system** that requires modernization to seamlessly integrate with enterprise applications and handle required data exchanges.

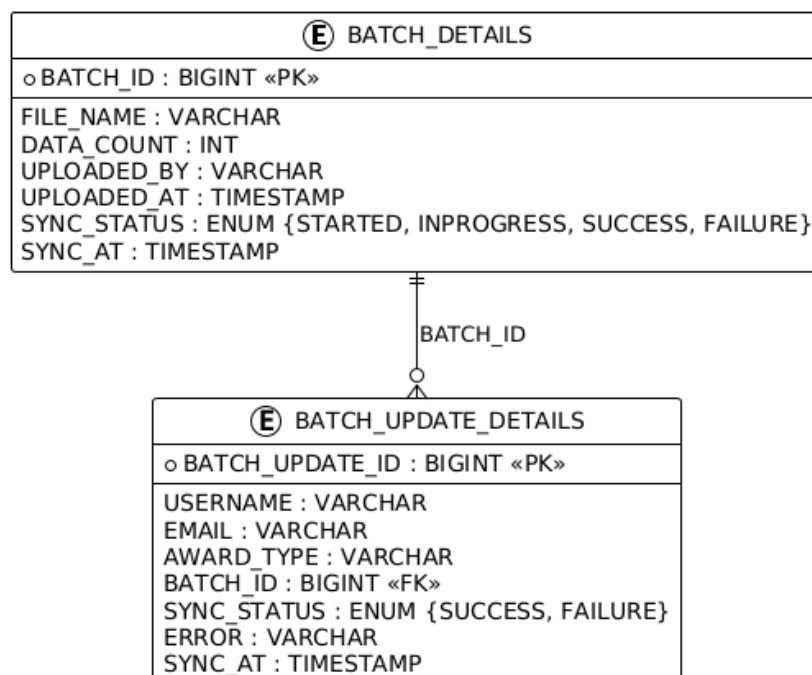
Proposed Automated Solution:

- Prepare award-winning candidate data in an Excel sheet and upload via UI.
- Stream uploaded data and persist with batch_id in the database.
- Enable RainFocus OAuth support for secure integration.
- Generate and cache OAuth tokens; renew automatically upon expiry.
- Invoke RainFocus APIs to update details, track each transaction status, and log results in the application (considering RainFocus restrictions).
- Persist update statuses in the database.
- Update the RainFocus status column (SUCCESS/FAILURE) in the Excel sheet.
- Send notification emails with the updated Excel attachment.
- Provide a dashboard to monitor batch-wise RainFocus sync status.
- Enable dashboard functionality to fetch historical sync details from RainFocus into Excel.

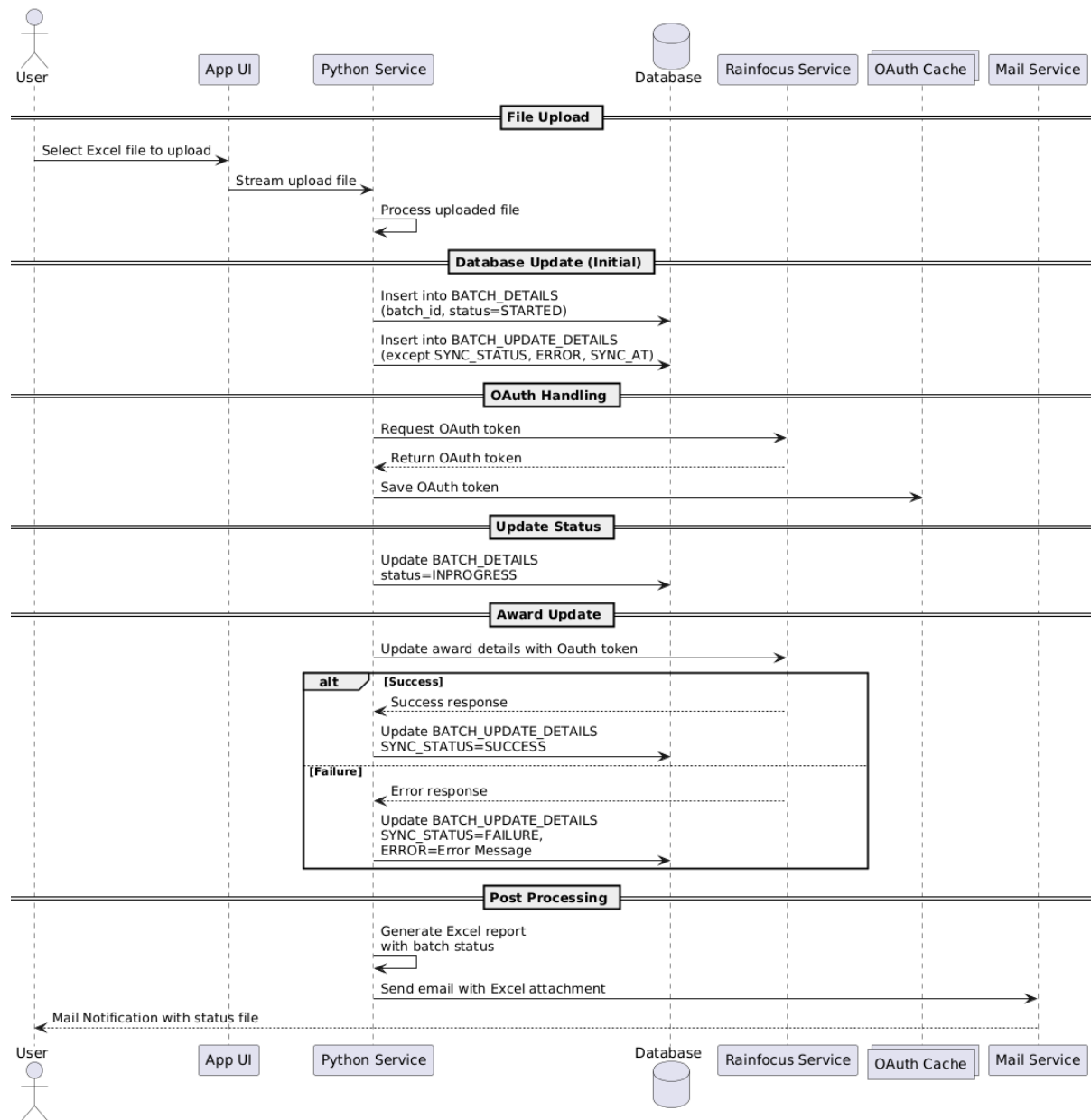
RainFocus Limitations:

- Cannot handle multiple concurrent requests.
- Update API does not support bulk payloads; only single-record updates are allowed.

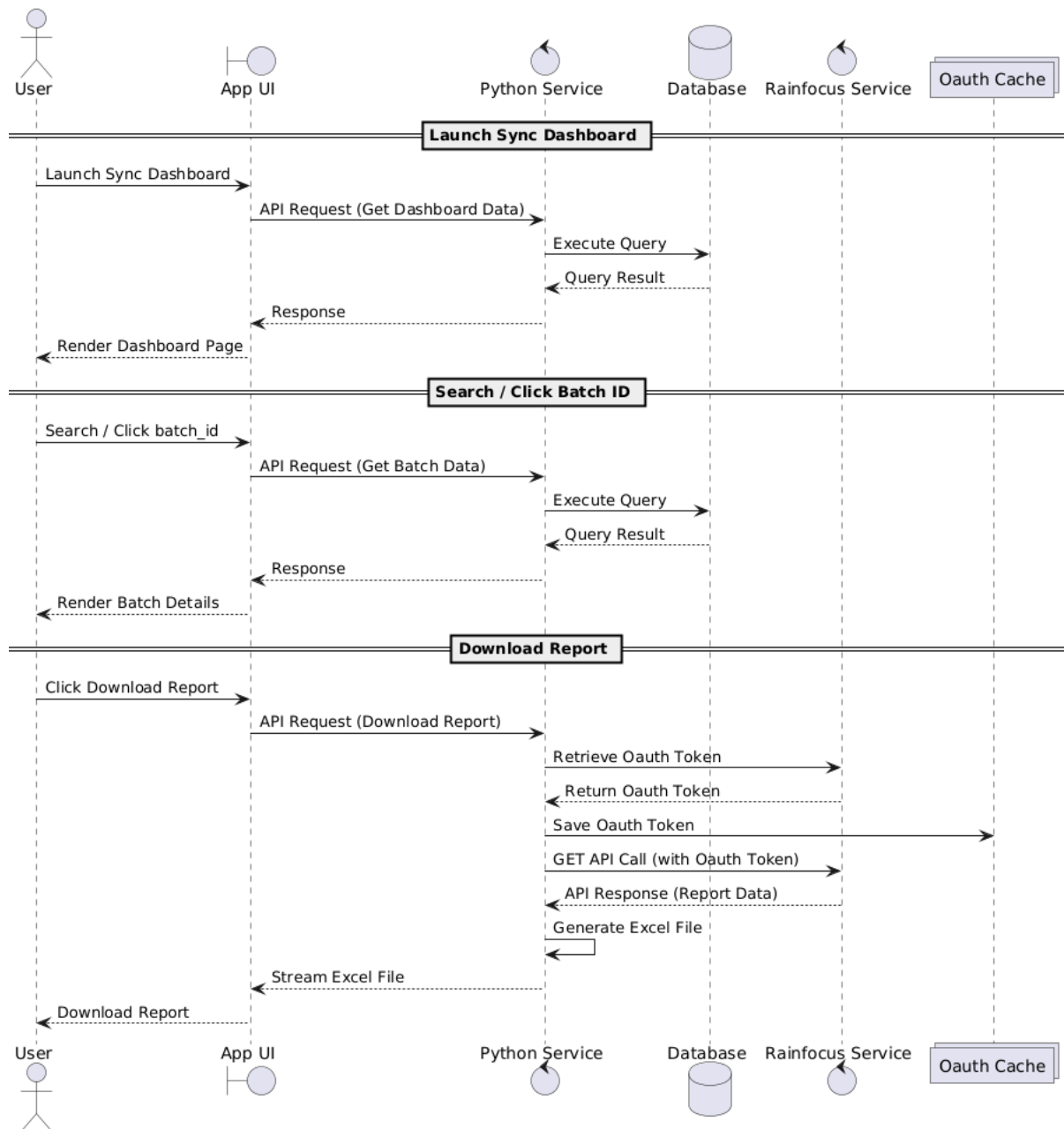
The diagram below illustrates the table structure and the modifications made to the existing table to accommodate the design changes.



The below diagram illustrates how the award Excel file is uploaded, processed by the system, and updated into a newly created table. It also depicts success and failure scenarios, followed by a notification sent to the user with a status Excel sheet attached.



The diagram below illustrates how the sync dashboard operates, including its functions, and explains how to download the latest sync details for comparison and status validation.



10. Optimization of Static File Management in Frontend Applications

Problem Statement:

- Currently, all static assets (videos, PDFs, Excel files, etc.) are stored directly in the frontend code repository. This approach:
- Increases the Docker image size, impacting pod replication in production.
- Consumes unnecessary compute and storage resources, raising operational costs.
- Requires frontend code deployment for every new or updated file (e.g., Excel reports), creating inefficiency.

Proposed Solution (Operational & Process):

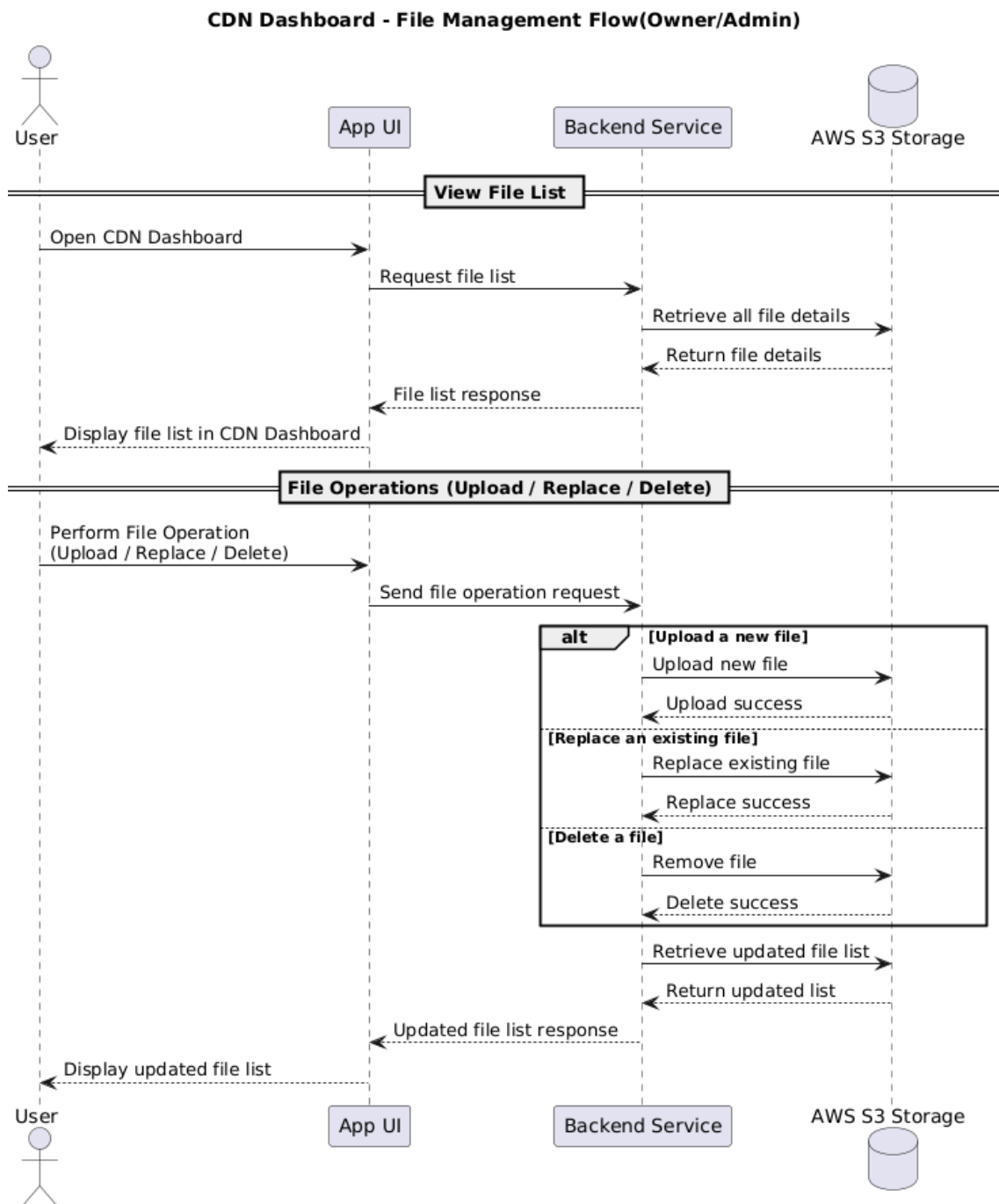
- Introduce a dedicated cloud storage service for managing static files.
- Remove static files from the frontend codebase.
- Configure storage details in the .env file.

- Enable the frontend to access static files via backend API calls.
- Backend retrieves files from cloud storage and returns them to the frontend.
- Integrate a CDN-backed dashboard for file upload, management, and updates.
- Allow new or updated files to be uploaded directly to cloud storage via UI.

Key Benefits:

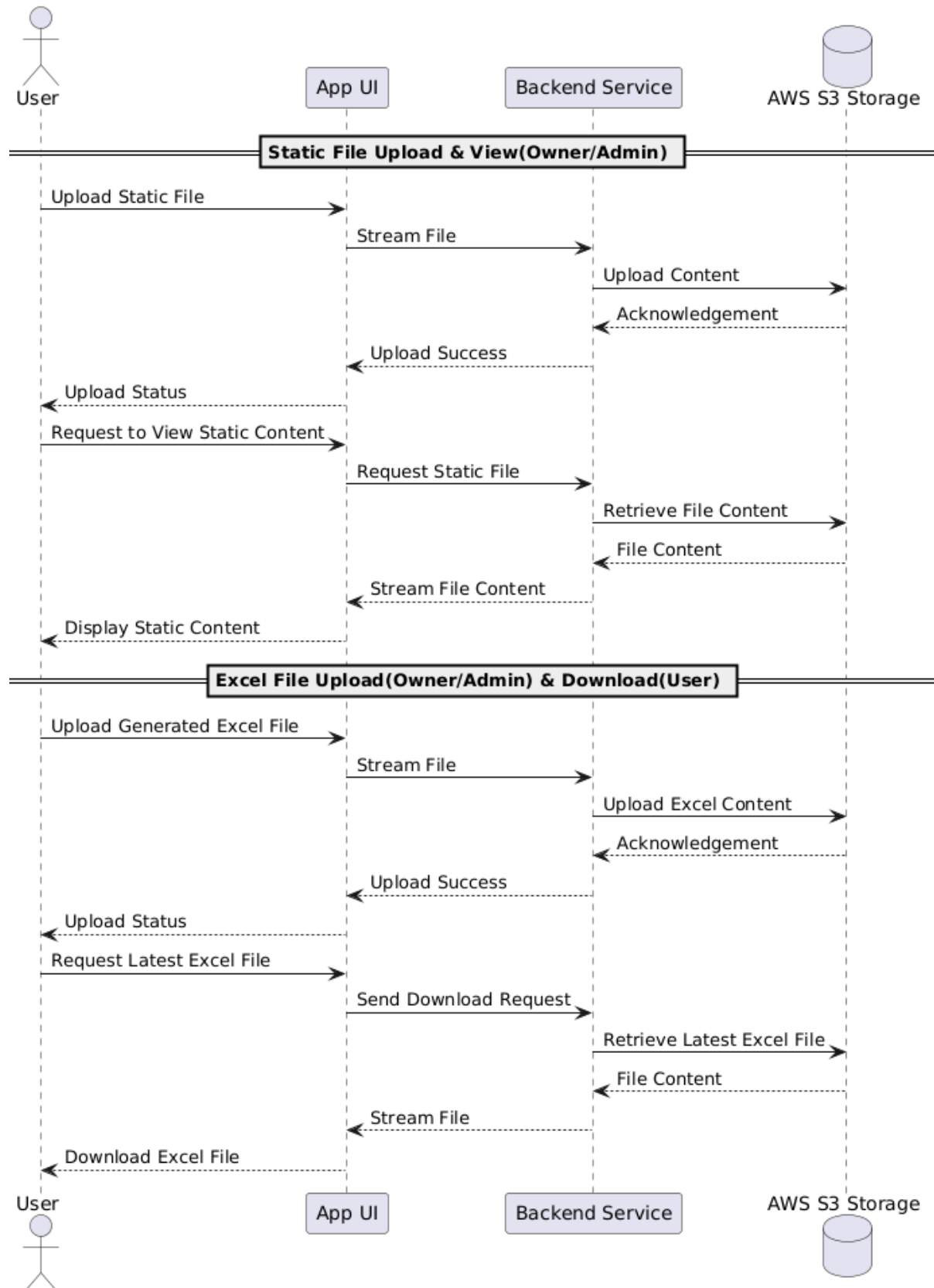
- Eliminates the need to modify and redeploy frontend code for file updates.
- Significantly reduces frontend Docker image size, improving scalability.
- Provides a user-friendly dashboard for complete static file lifecycle management.
- Supports all file types with seamless upload and update capabilities.
- Reduces infrastructure resource usage and lowers operational costs.

The diagram below illustrates the CDN dashboard workflow and how various operations are performed through it.



The below diagram illustrates the process of moving static files to an Amazon S3 bucket, supporting file uploads and retrievals. It also shows how the latest files can be uploaded via the CDN dashboard, eliminating the need for redeployment.

File Upload and Download Flow (Static & Excel Files)



11. Requirement Specification: Document Upload and Review Workflow

New Requirement:

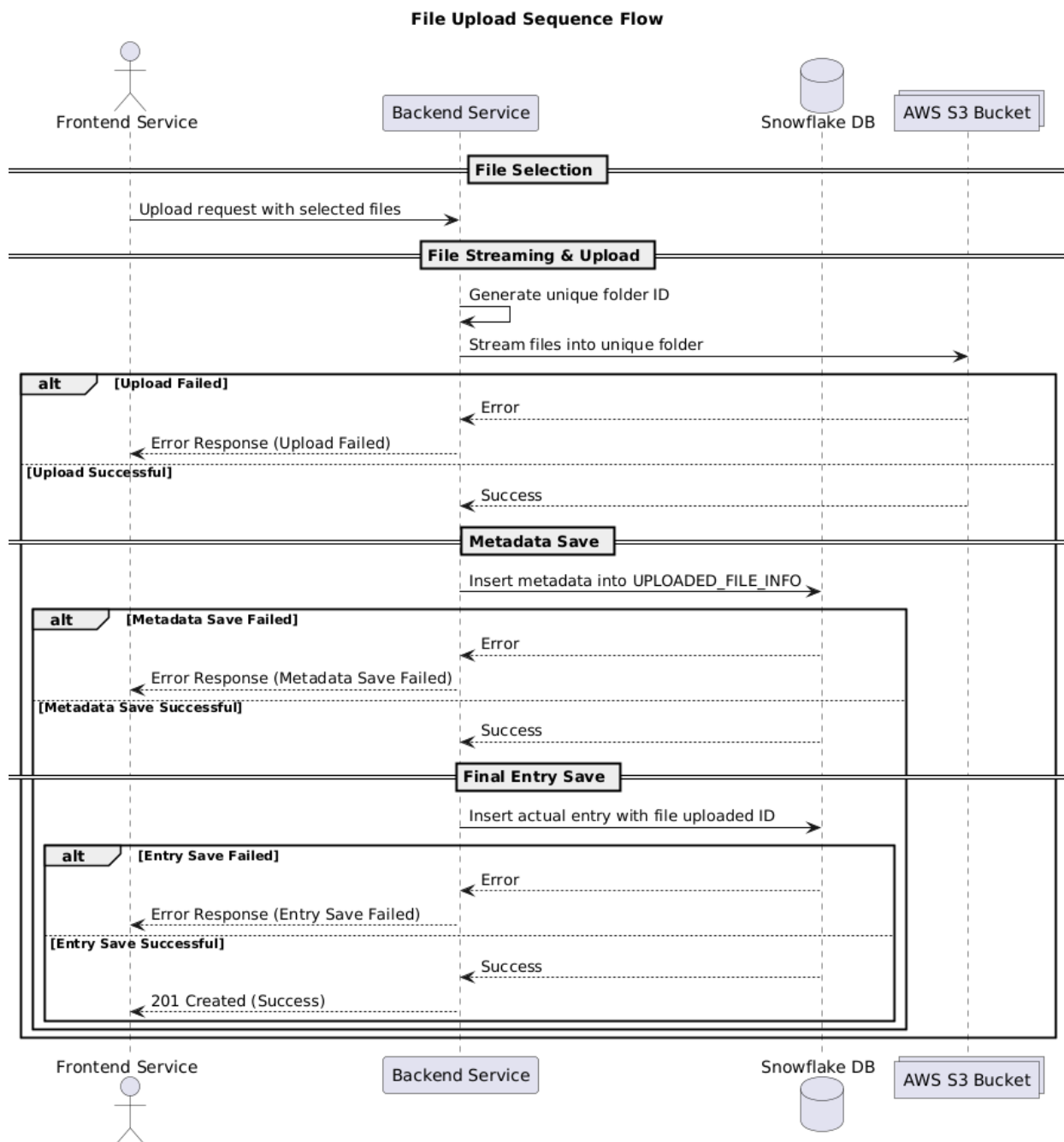
Enable support for uploading and attaching documents in the following formats to newly created entries, allowing reviewers to view the documents prior to approval:

- **Supported File Types:** .pdf, .xlsx, .csv, .docx, .pptx, .png, .jpg, .jpeg

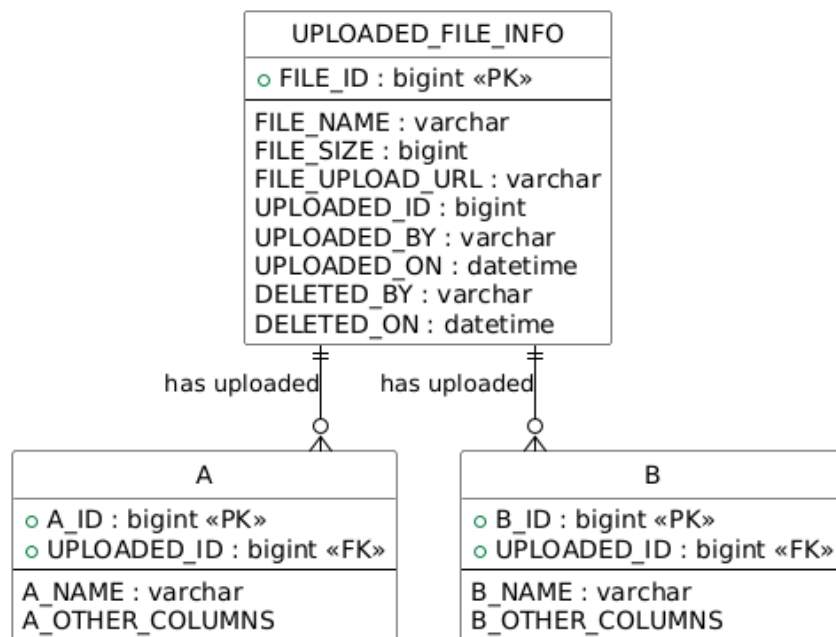
Tools/Storage:

- **Cloud Object Storage:** AWS S3 bucket

The sequence diagram below illustrates the end-to-end document upload, review, and approval process.



The ER diagram below illustrates the newly introduced table *UPLOADED_FILE_INFO* and its relationships with the existing tables *A* and *B*.



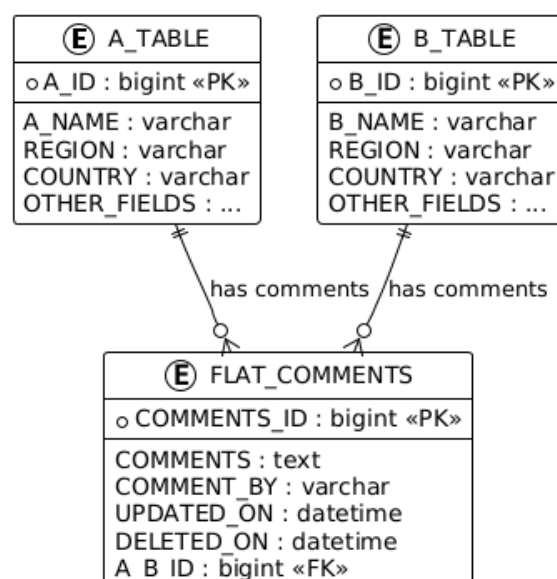
12. Requirement Change: Hierarchical Comment Tracking & API Optimization

New Requirement:

Previously, comments were stored as flat messages, only showing the author and date. The new requirement introduces hierarchical tracking, where replies are linked to specific comments, maintaining a structured comment history.

- The hierarchy applies **only to the current period**, while the previous period continues to use the flat table design.
- Certain attributes in the API response, unnecessary for NGCGI integration, should be removed for the current period but retained for historical data.

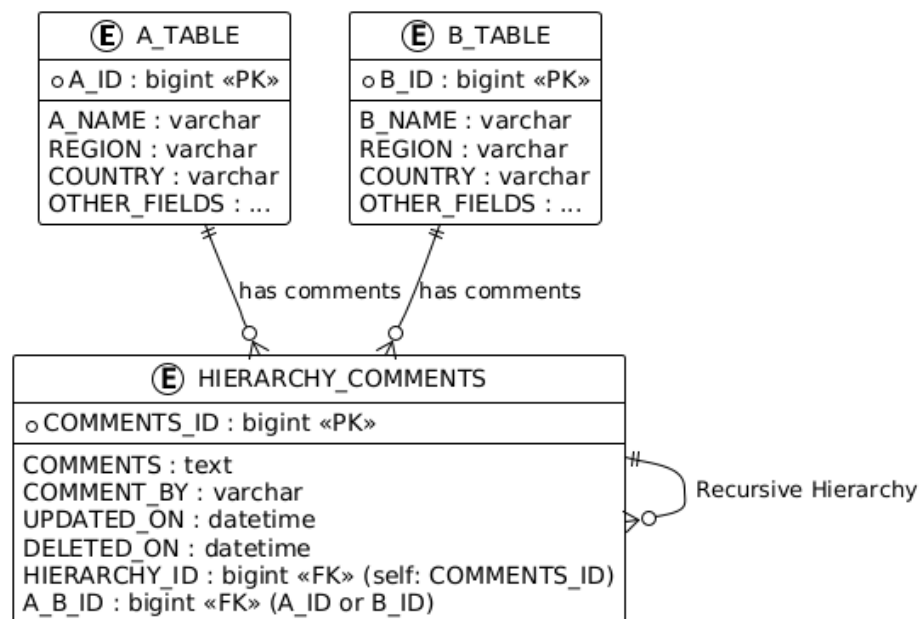
The ER diagram below illustrates the tables that store flat comments.



Proposed Solution:

- Redesign the table to support hierarchical comment history.
- Implement **self-joins** to efficiently retrieve nested comment structures.
- Develop a new API to fetch hierarchical comments.
- Remove redundant attributes from current-period responses while preserving previous-period responses unchanged.
- Introduce **API versioning** to differentiate responses based on the selected period.

The ER diagram below illustrates the extended model for storing comments in a hierarchical structure.



13. Optimizing Branching Strategy for CI/CD and Environment Management

Problem Statement:

Currently, the CI/CD pipeline is tied only to the release branch. To test changes in the development environment, changes must first be merged into the release branch, then reverted after testing. This creates risks for urgent production fixes, forces dependency on the release branch, and blocks development during testing cycles.

Proposed Solution:

- Introduce a **develop branch** for all ongoing development activities.
- Create a **dedicated CI/CD pipeline** for the develop branch.
- Commits to the develop branch generate artifacts and deploy them directly to the **development environment**.
- Maintain the **release branch** exclusively for production-ready code and urgent hotfixes.

Checklist:

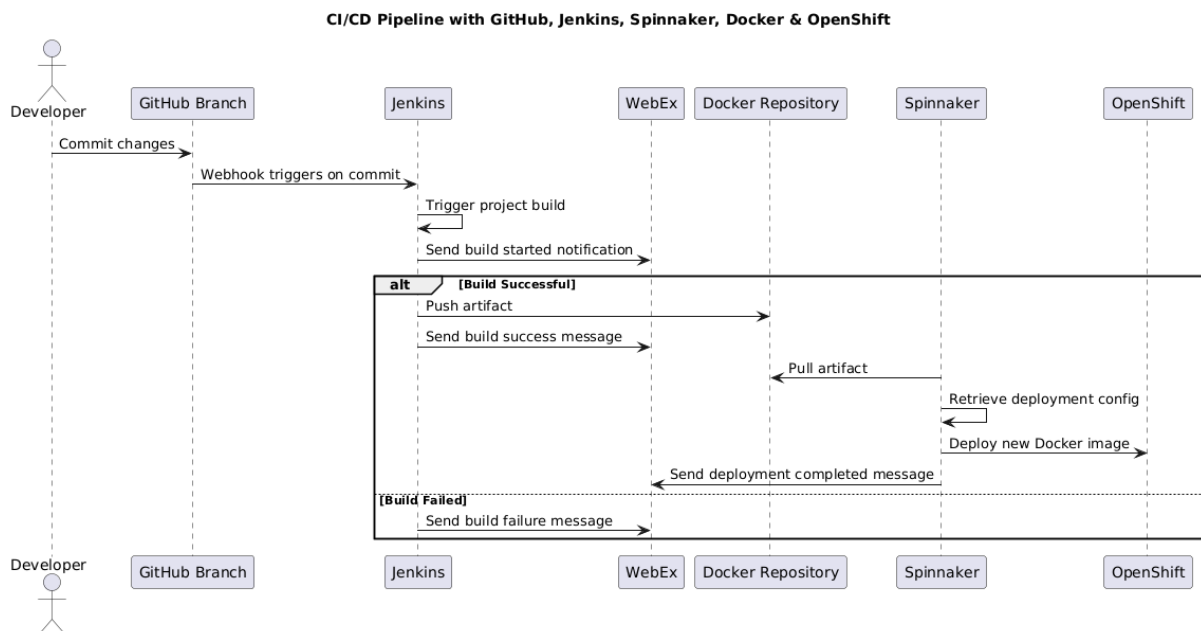
- Production deployments occur **only via the release branch**.
- Any changes in the release branch are merged back into the develop branch.

- Artifacts from the release branch must be explicitly selected before promoting to higher environments.
- During production releases, commits to the develop branch are paused to avoid conflicts.
- Testing teams verify correct artifacts are deployed and validated.

Benefits:

- Prevents development environment from being blocked during testing.
- Eliminates effort and risk of reverting test changes.
- Enables continuous development and testing without impacting release stability.
- Facilitates faster switching between branches for efficient testing.
- Allows development and testing to run **in parallel**, improving productivity and reducing wait times.

The sequence diagram below illustrates the CI/CD pipeline configuration and the deployment of changes across environments.



14. Optimizing Rate Details Management and Query Performance

Problem Statement:

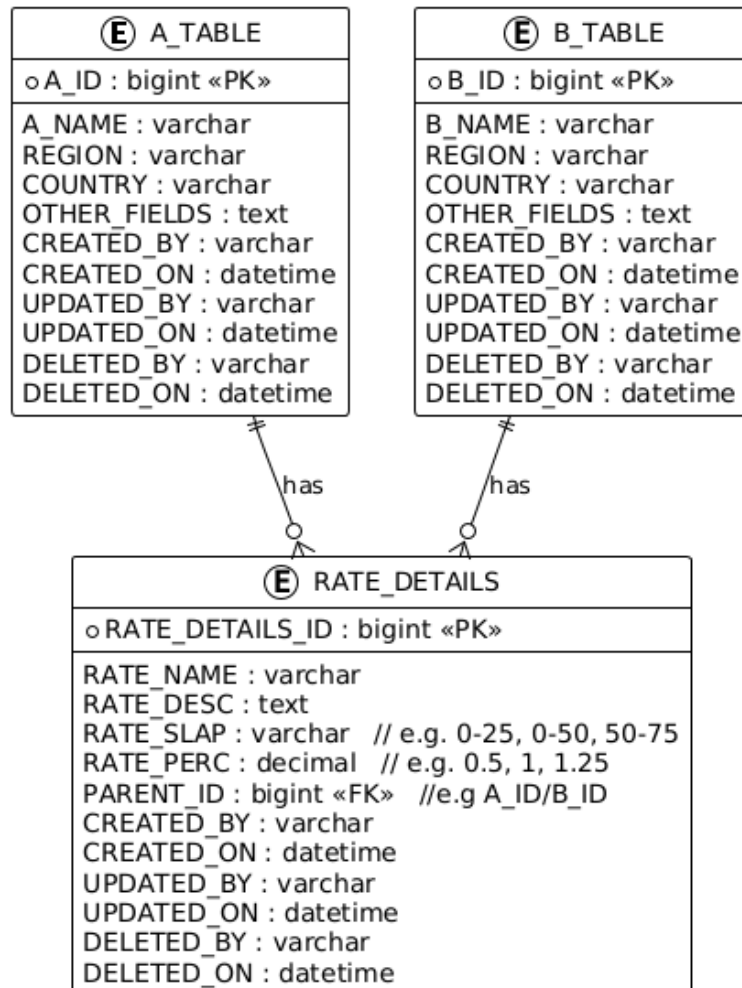
The parent table currently contains extensive rate details, leading to duplicate row entries and making it difficult to manage over time. Frequent changes to rate details require updates across multiple columns, increasing complexity, reducing maintainability, and slowing down query performance.

Solution (Performance Oriented):

- Normalize the schema by moving rate details into a dedicated table **RATE_DETAILS** with a reference to the parent ID.
- Ensure all modifications to rate details are handled exclusively in the **RATE_DETAILS** table.

- Eliminate duplicate entries from the parent table to improve data integrity and query performance.

The ER diagram below illustrates the table-level changes made through normalization to eliminate duplicates and improve query performance.



15. Application Usage Monitoring and Metrics Dashboard

Problem Statement:

The application requires tracking of key usage metrics such as login activity, specific page access, and role-based actions. These metrics need to be shared with management to analyse application usage and effectiveness. However, the external application does not provide accurate metrics, forcing the team to rely on Splunk logs. This approach is manual, effort-intensive, and challenging when details are required on demand.

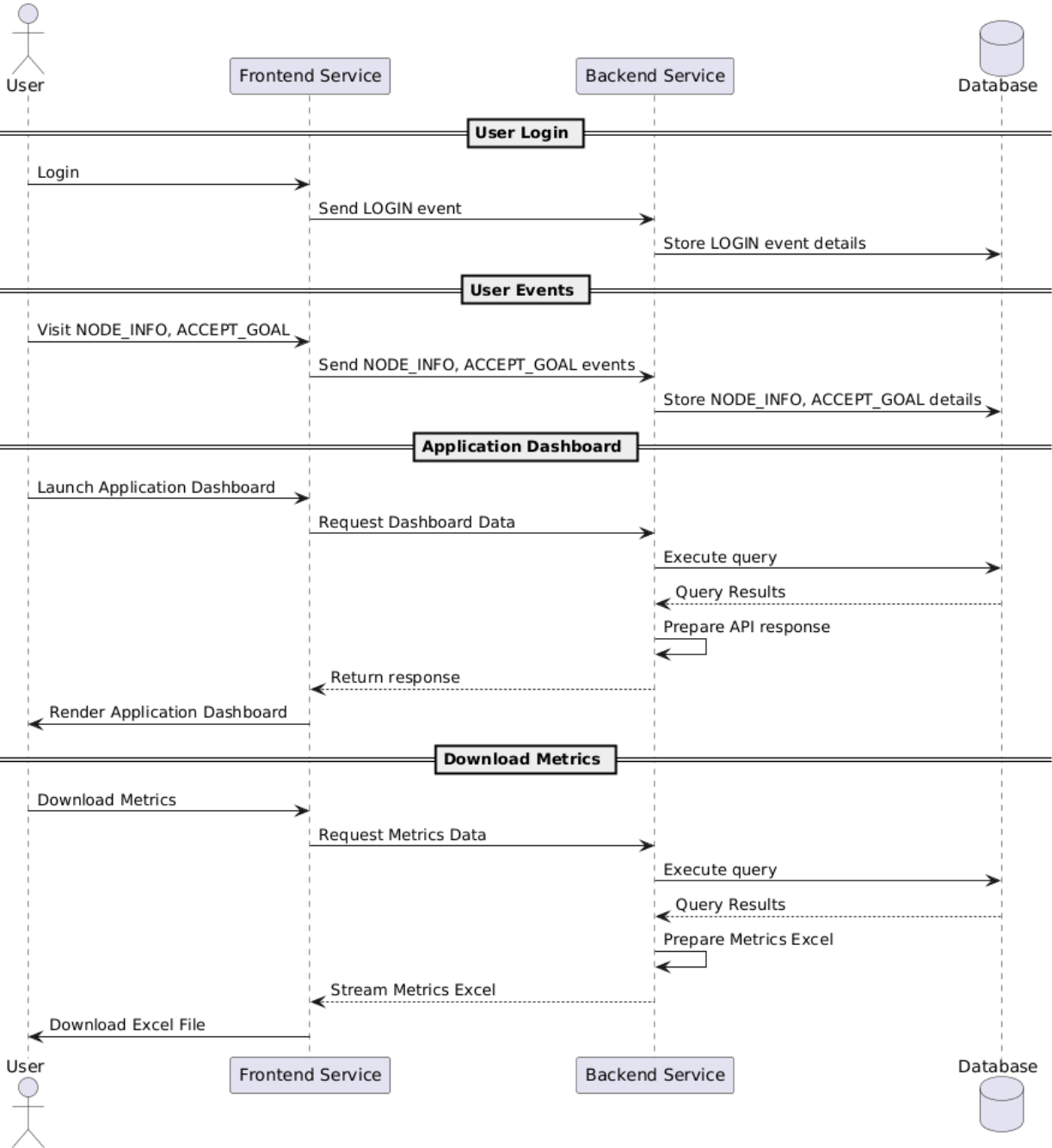
Proposed Solution:

- Store login and role-based activity metrics directly in the database.
- Develop a monitoring dashboard to provide a complete view of application metrics.
- Enable filters for detailed insights and support report downloads.
- Provide tracking on multiple timeframes (last visit, daily, weekly, monthly, yearly).

The ER diagram below represents the table structure for storing event data and associated metrics used in the application dashboard.

E APP_METRICS	
o APP_METRICS_ID : BIGINT «PK»	
USER : VARCHAR(100)	
PROXY_USER : VARCHAR(100)	
EVENT : VARCHAR(50) «ENUM: LOGIN, NODE_INFO, ACCEPT_GOAL»	
ROLE : VARCHAR(50)	
LAST_VISITED_ON : TIMESTAMP	

The sequence diagram below illustrates the workflow for storing events, displaying metrics on the application dashboard, and downloading metrics from the dashboard.



16. External Application Integration via Secure API Proxy

Problem Statement:

The current approach uses an IFRAME to embed an external application for accessing a specific page. However, this introduces several issues:

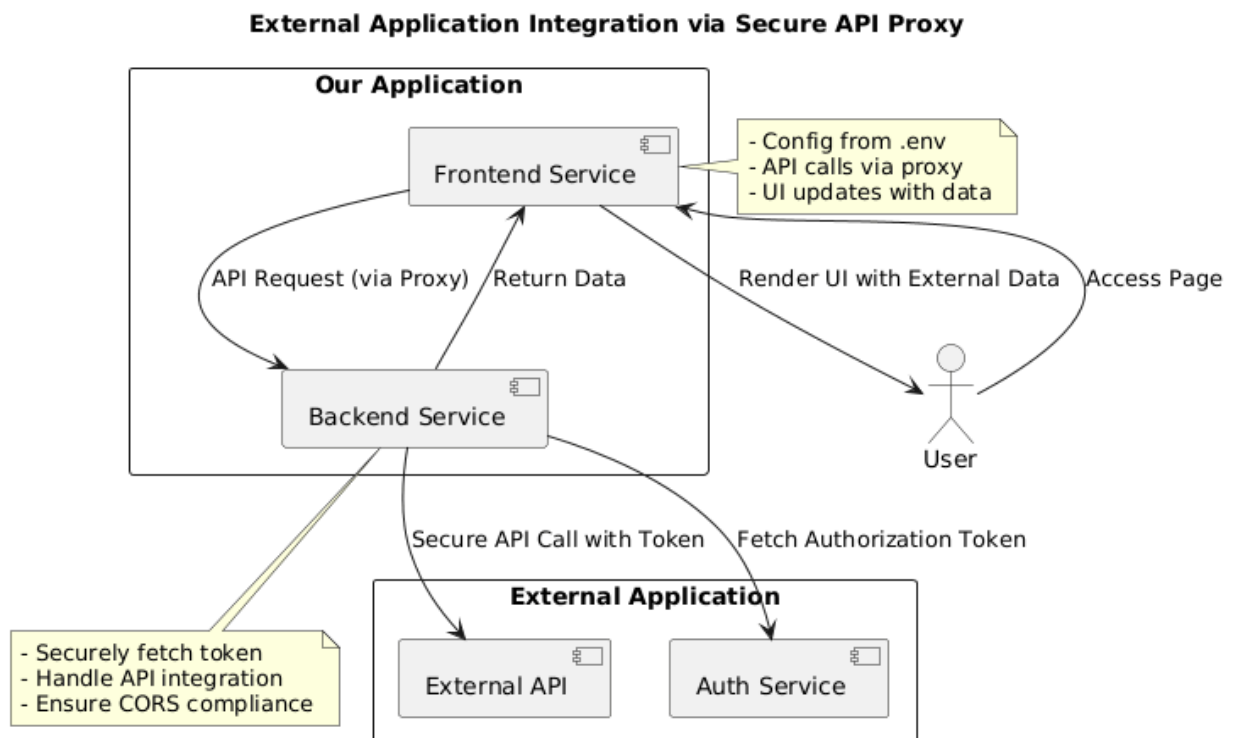
- The external application has its own authentication and authorization, requiring a separate login flow.
- Browsers block cross-domain access through iframes due to security restrictions (CORS and zero-trust policies without VPN).
- The login process redirects users to the external application's home page instead of the intended route.

Proposed Solution:

To enable secure integration while avoiding iframe and CORS issues:

- Define external application configuration in .env.
- Implement a backend service to fetch the authorization token and call the external application's APIs.
- Use the frontend service to interact with the backend via a proxy.
- Update the frontend UI to display the external application data seamlessly within our application.

The following architecture diagram illustrates the integration workflow with the external application and clearly defines the associated roles and responsibilities.



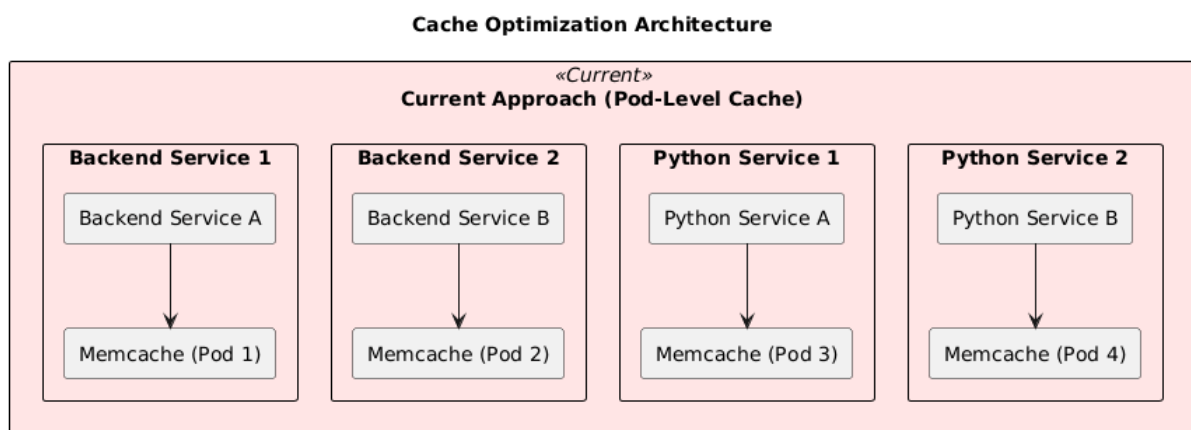
17. Centralized Cache Optimization with Redis

Problem Statement:

The current implementation uses application-level Memcache at the pod level to improve performance. However, this approach introduces inefficiencies:

- Cache data is maintained separately in each pod, leading to duplication.
- Every pod must recheck and recreate cache entries if they don't exist.
- When pods are replicated, newly created pods also maintain their own duplicate cache, further increasing overhead.

The following architecture diagram illustrates the pod-level cache, showing that as the number of pods increases, the corresponding memcache also scales at the pod level.

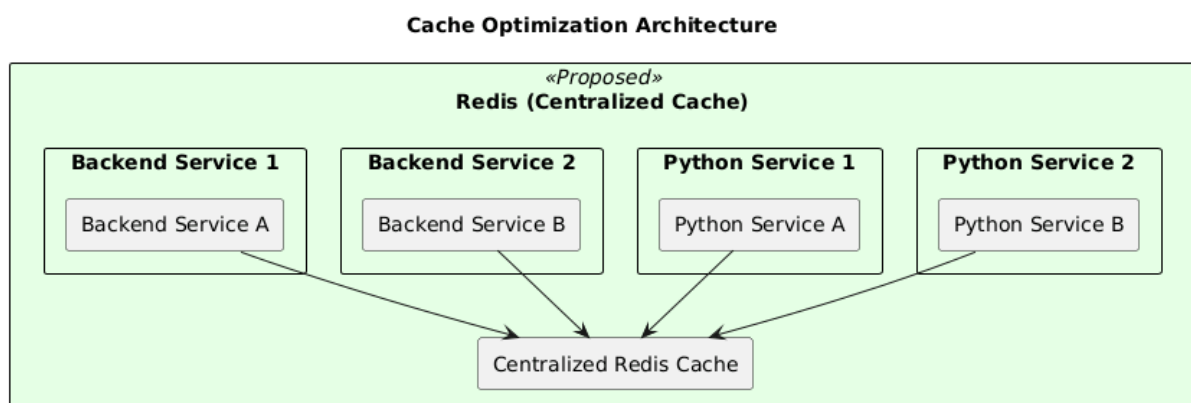


Proposed Solution:

Replace pod-level Memcache with a centralized Redis cache. Redis ensures:

- Elimination of duplicate caches across pods.
- A single, centralized cache accessible by all pods.
- Seamless scaling, as new pods can directly leverage existing cached data without maintaining their own local copies.

The below architecture diagram illustrates the improvement of introducing a centralized Redis cache, enhancing performance and eliminating the need for pod-level caching.



Note: Be mindful when clearing cache to prevent data inconsistency and unnecessary authentication calls when accessing or integrating with external APIs.

18. Cost Optimization and Technology Modernization Initiatives

1. Migration of Third-Party Application (Completed)

Successfully migrated the Media Hub third-party application to Cisco on-premises as a modern web application using the latest technology, with deployment to the cloud environment.

- Enables full source code management and seamless adoption of company-level infrastructure standards.
- Delivers higher customer value while eliminating third-party dependency.
- Achieved significant cost savings by consolidating operations with minimal overhead.

2. Python Automation & Apache Airflow (In Progress)

Initiated scheduled Python-based automation to replace costly Alteryx workflows.

- Reduced licensing expenses by automating processes via Python.
- Exploring Apache Airflow as a free alternative for remaining workflows.
- Shared documentation for local setup and feature exploration.

3. Redis Cache Integration (In Progress)

Optimizing Snowflake database query performance and reducing operational costs.

- Implementing Redis cache to store frequently used non-modifiable data.
- Improves efficiency for long-running queries while lowering compute costs.

4. Application Monitoring Enhancement (In Progress)

Replaced third-party monitoring tools with in-application dashboards.

- Provides real-time application statistics, metrics tracking, and reporting capabilities.
- Eliminated external monitoring costs.

5. Bulk Email Support (Completed)

Integrated real-time email communication support directly within the application.

- Removed dependency on external bulk email providers.
- Ensures faster delivery of pending and failed emails, reducing communication overhead.

19. Conclusion – Enterprise-Grade Application Transformation

The application has successfully evolved into an enterprise-standard platform with the following capabilities:

- Modular, scalable, and maintainable codebase
- Secure authentication and authorization with role-based access control (RBAC)
- API-level RBAC for enhanced security
- Database-level monitoring
- Seamless access with or without VPN
- Centralized logging support
- Real-time communication via WebEx and email
- CDN-enabled static file storage and management with dashboard control
- Application-level monitoring, analytics dashboard, and downloadable reports
- Integration with external applications
- Centralized Redis cache support
- Reduced operational costs by eliminating external dependencies